Design Documentation

for

WhatsApp Chatbot

Version 1.1

Prepared by

GROUP: Mon 6-8 Grp 4

Gabria Gordon	**620173389**	**gabria.gordon@mymona.uwi.edu**
Joshua-Amani Green	**620172653**	**joshuaamani.green@mymona.uwi.edu**
Patrick Marsden	**620169874**	**patrick.marsden@mymona.uwi.edu**

Course Instructor: **Claudia Allen**
Course: **COMP2171**
Studio Facilitator: **Aldane Milton**
Date: **April 10, 2026**

TABLE OF CONTENTS

1 Background and Proposed Solution 1
1.1 Background and Proposed Solution 1
1.2 Background 1
1.3 Proposed Solution 1
2 Domain Model 1
2.1 Domain Model 1
3 Use Case Model 3
3.1 Use Case Diagram 3
3.2 Use Case Specifications & Activity Diagrams 4
3.2.1 Check Order Status 4
3.2.2 Schedule Appointment 5
3.2.3 Receive Reminders 7
3.2.4 View Dashboard 9
3.2.5 Update Pricing 11
3.2.6 Update Job Status 13
3.2.7 Send Completion Message 15
3.2.8 Send Proactive Notification 17
3.2.9 User Interface 19
3.2.10 Receive Report 21
3.2.11 Edit Booking 23
4 Architectural Design 25
4.1 Package Diagram 25
4.2 Component Diagram 26
4.3 Deployment Diagram 27
5 Detailed Design 28
5.1 Class Diagram 28
5.2 Sequence, Object & State Diagrams 28
5.2.1 Check Order Status 28
5.2.2 Schedule Appointment 29
5.2.3 Receive Reminders 31
5.2.4 View Dashboard 33
5.2.5 Update Pricing 35
5.2.6 Check Order Status 35
5.2.7 Check Order Status 36
5.2.8 Check Order Status 36
5.2.9 Check Order Status 36
5.2.10 36
5.2.11 36

6 Implementation **36**
6.1 Discussion 36
6.2 Code Repository 38
Code Repository 38

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0			
1.1	Patrick Marsden	Final Update	4/10/2026

1 Background and Proposed Solution

1.1 Background and Proposed Solution

1.2 Background

White Rose Interiors currently handles customer orders and quotations manually through WhatsApp messages. This leads to inefficiencies such as delayed responses, difficulty tracking orders, and lack of centralised data management. Customers often need to repeatedly follow up for updates, and staff must manually search conversations to retrieve order details.

1.3 Proposed Solution

The proposed system is a WhatsApp-based chatbot integrated with a web-based admin dashboard. The chatbot automates customer interactions such as requesting quotations, placing orders, and checking order status.

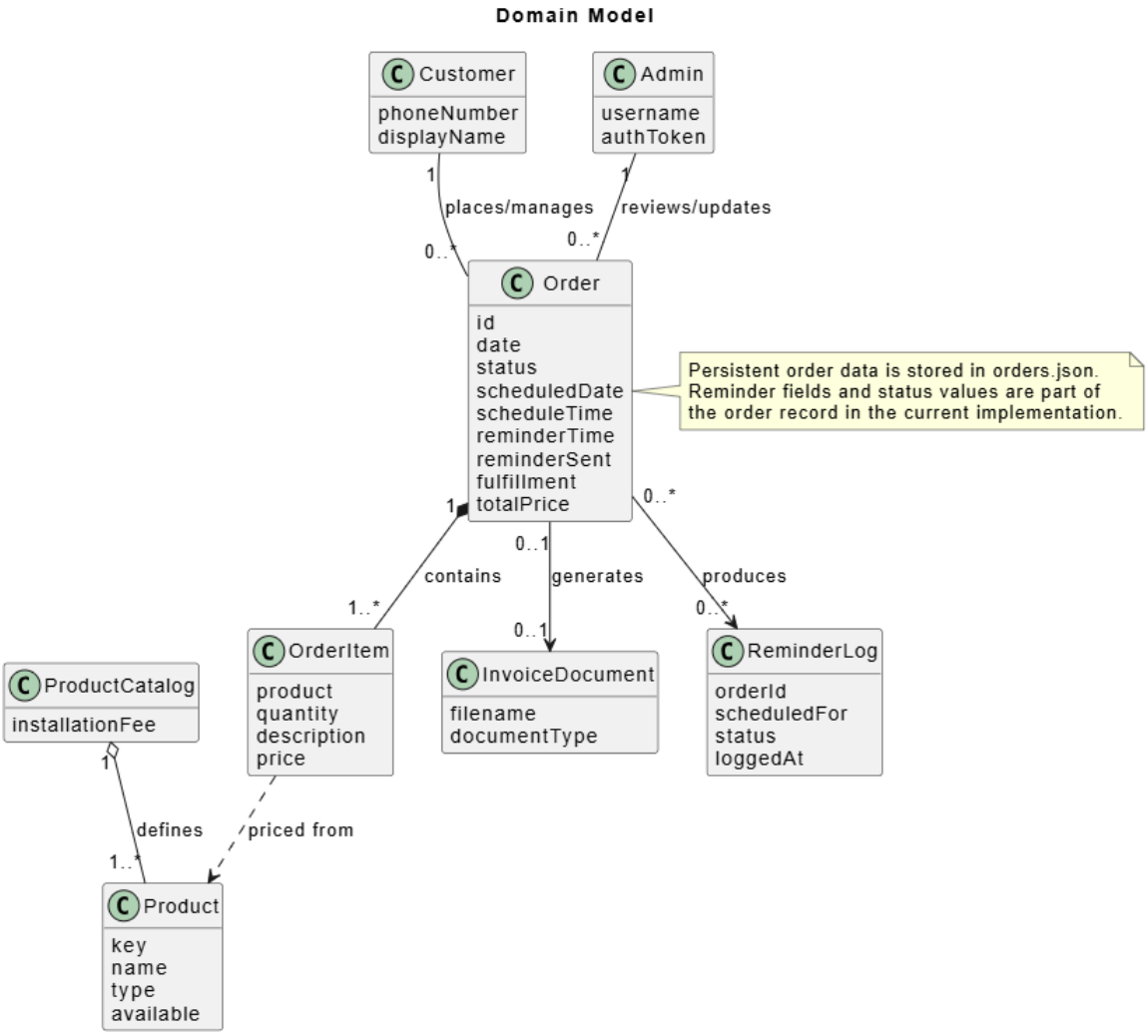
The system will:

- Automatically respond to customer queries via WhatsApp
- Store order data in a structured format (JSON)
- Generate PDF estimates and invoices
- Provide an admin dashboard for managing orders and viewing analytics

This improves efficiency, reduces manual work, and enhances customer experience.

2 Domain Model

2.1 Domain Model

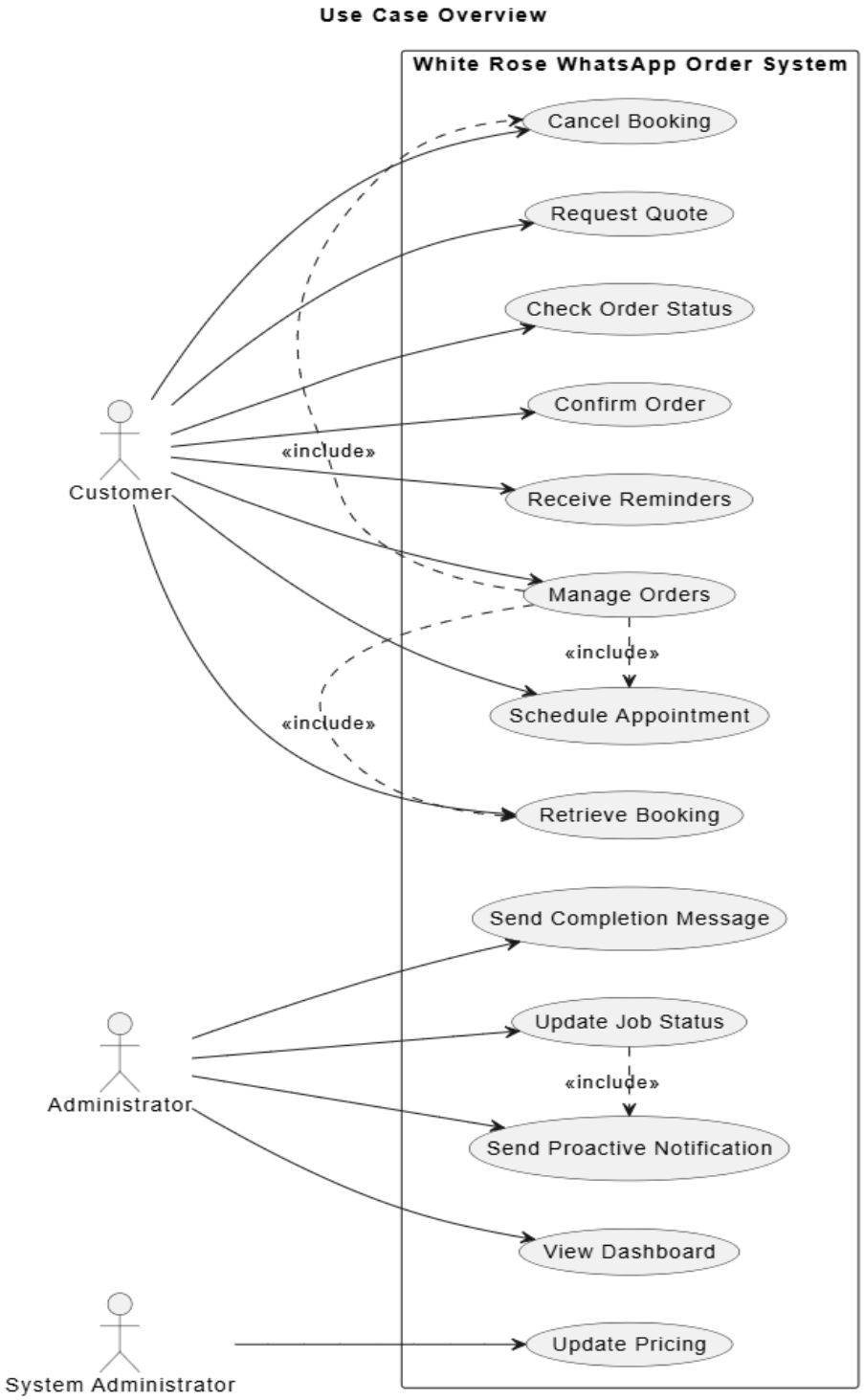


Domain Model Diagram

Shows the main business entities and relationships used in the system.

3 Use Case Model

3.1 Use Case Diagram

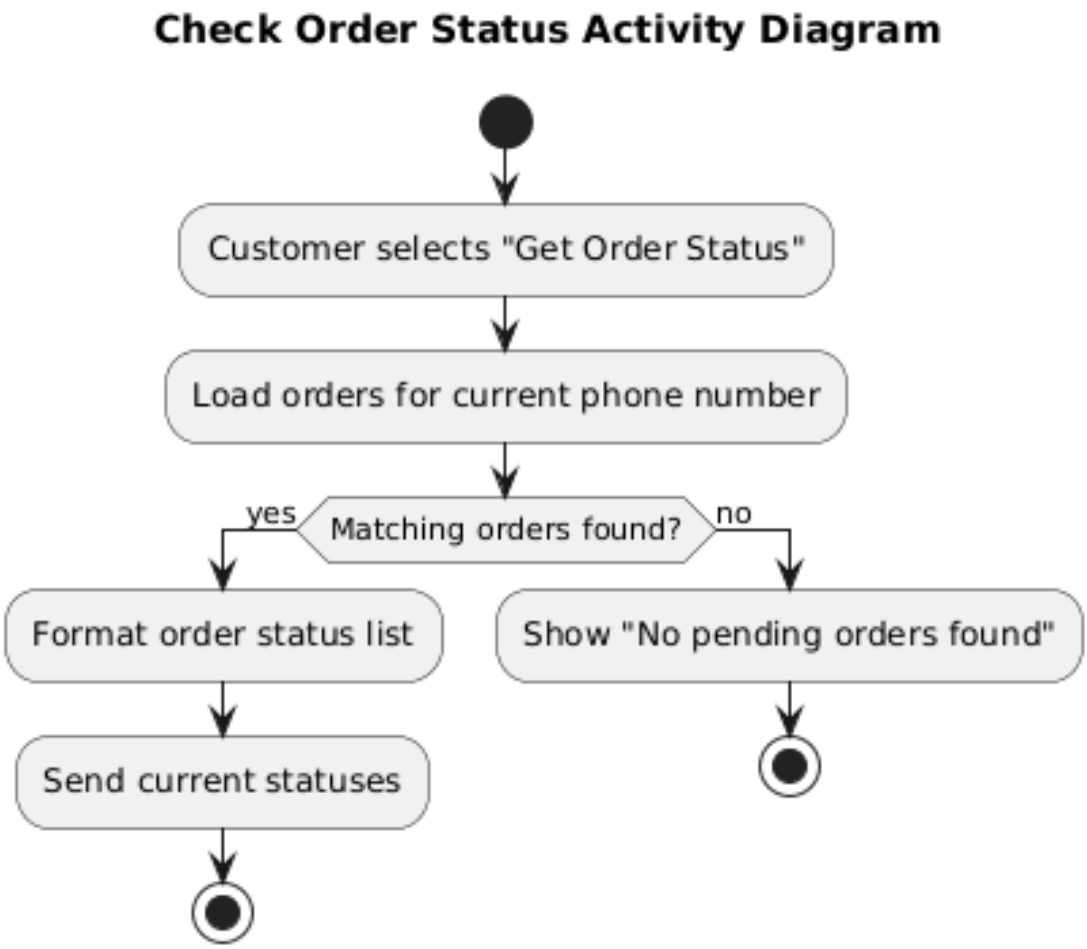


Use Case Overview Diagram

Shows how customers and administrators interact with the system.

3.1.1 Check Order Status

Use case name	**Check Order Status**	
Related Requirements	Requirement 5	
Goal in Context	A customer queries the current status of their order using their order number or phone number.	
Preconditions	The customer has a valid order number or registered phone number. The system has access to the orders database.	
Successful End Condition	The customer is presented with the current status of their order.	
Failed End Condition	The system cannot find a matching order record and informs the customer.	
Primary Actors	Customer	
Secondary Actors	Orders Database	
Trigger	The customer submits their order number or phone number to query their order status.	
Included Cases	Fetch Job Status	
Main Flow	*Step*	*Action*
	1	*The system prompts the customer to enter their order number or phone number.*
	2	*The customer enters their order number or phone number.*
	3 include::Fetch Job Status	*The system retrieves the status of the job from Orders Database.*
	4	*The system displays the current job status to the customer.*
Extensions	*Step*	*Branching Action*
	2.1	*The customer enters an invalid or unrecognised identifier.*
	2.2	*The system notifies the customer that no matching order was found.*

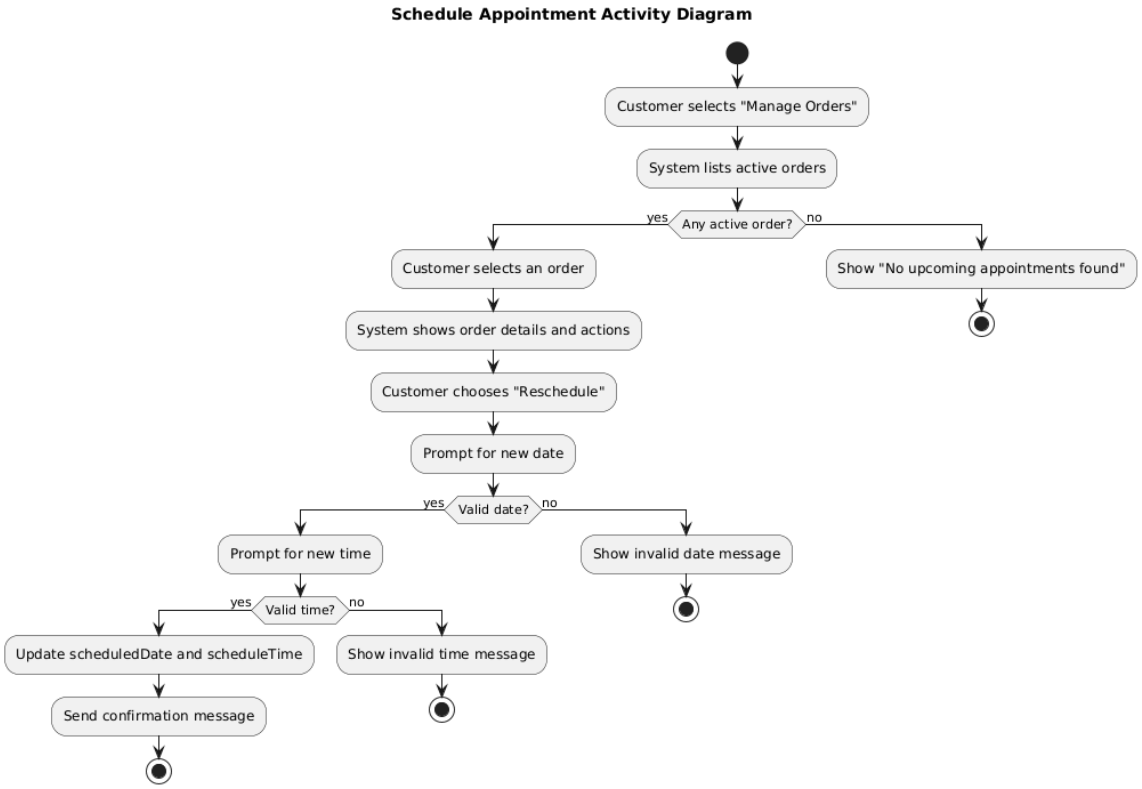


Check Order Status Activity Diagram

This activity diagram illustrates the workflow for the check order status use case.

Use case name	**Check Order Status**	
Related Requirements	Requirement 2	
Goal in Context	A customer schedules an appointment for a service (e.g., consultation or measurement) through the WhatsApp chatbot.	
Preconditions	The customer is interacting with the system via WhatsApp. The system has access to available appointment slots.	
Successful End Condition	The appointment is successfully scheduled and stored in the system, and the customer receives a confirmation message.	
Failed End Condition	The appointment is not scheduled due to invalid input or unavailable time slot, and the system informs the customer.	
Primary Actors	Customer	
Secondary Actors	Appointments Database	
Trigger	The customer requests to schedule an appointment.	
Included Cases	Check Availability	
Main Flow	*Step*	*Action*
	1	*The system prompts the customer to enter a preferred date and time for the appointment.*
	2	*The customer enters the appointment details.*
	3 include::Check Availability	*The system checks if the selected time slot is available.*
	4	*The system saves the appointment details in the database.*
	5	*The system sends a confirmation message to the customer.*
Extensions	*Step*	*Branching Action*

	2.1	*The customer enters an invalid or incomplete date/time.*
	2.2	*The system prompts the customer to re-enter valid details.*
	3.1	*The selected time slot is unavailable.*
	3.2	*The system asks the customer to choose another time slot.*



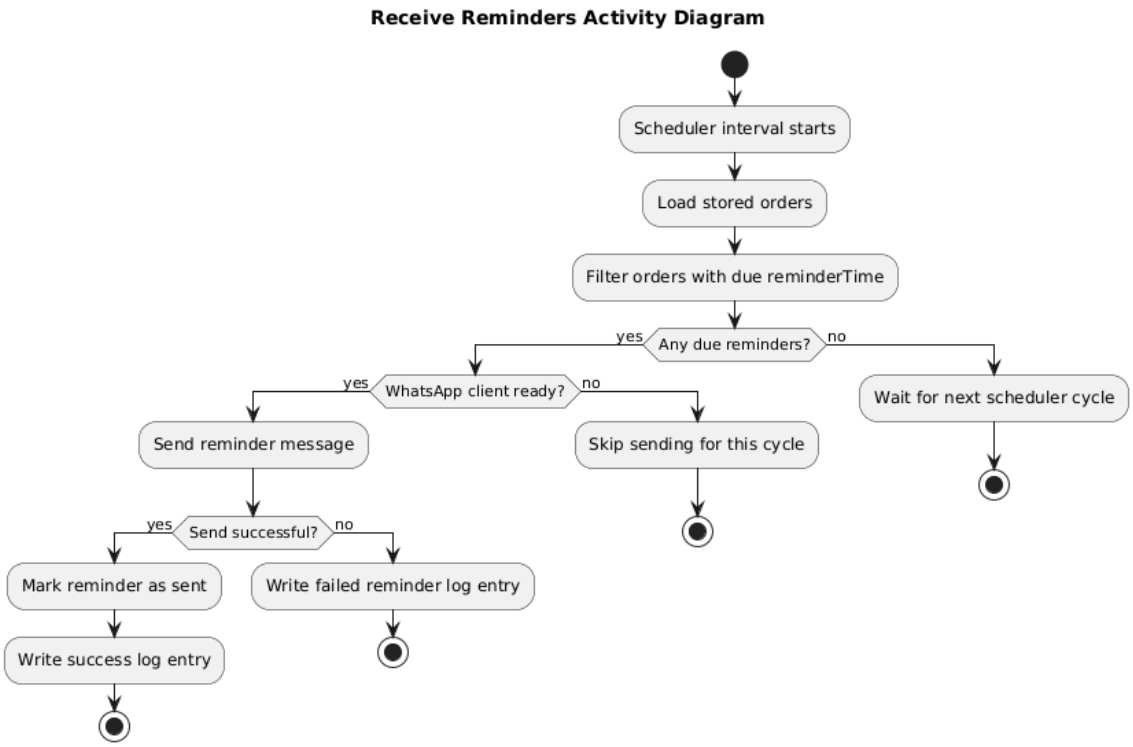
Schedule Appointment Activity Diagram

This activity diagram illustrates the workflow for the schedule appointment use case.

3.1.2 Receive Reminders

Field	Details
Use case name	**Receive Reminders**
Related Requirements	Requirement 6
Goal in Context	A customer receives an automated reminder for an upcoming appointment or scheduled job.
Preconditions	The customer has a scheduled appointment or job in the system. Reminder functionality is active and scheduled.
Successful End Condition	The reminder message is successfully sent to and received by the customer.
Failed End Condition	The reminder is not sent due to missing data or system failure, and the system logs the issue.
Primary Actors	Customer
Secondary Actors	Reminder Scheduler, Appointments Database
Trigger	The scheduled reminder time is reached.
Included Cases	Fetch Appointment Details
Main Flow	**Step 1:** The system detects that a reminder time has been reached. **Step 2:** *include::Fetch Appointment Details* The system retrieves the relevant appointment or job details. **Step 3:** The system generates a reminder message. **Step 4:** The system sends the reminder to the customer via WhatsApp. **Step 5:** The customer receives the reminder message.

Extensions	**Step 2.1:** No appointment or job is found for the reminder. **Step 2.2:** The system logs the issue and does not send a reminder. **Step 4.1:** The message fails to send due to network or API error. **Step 4.2:** The system retries or logs the failure for admin review.



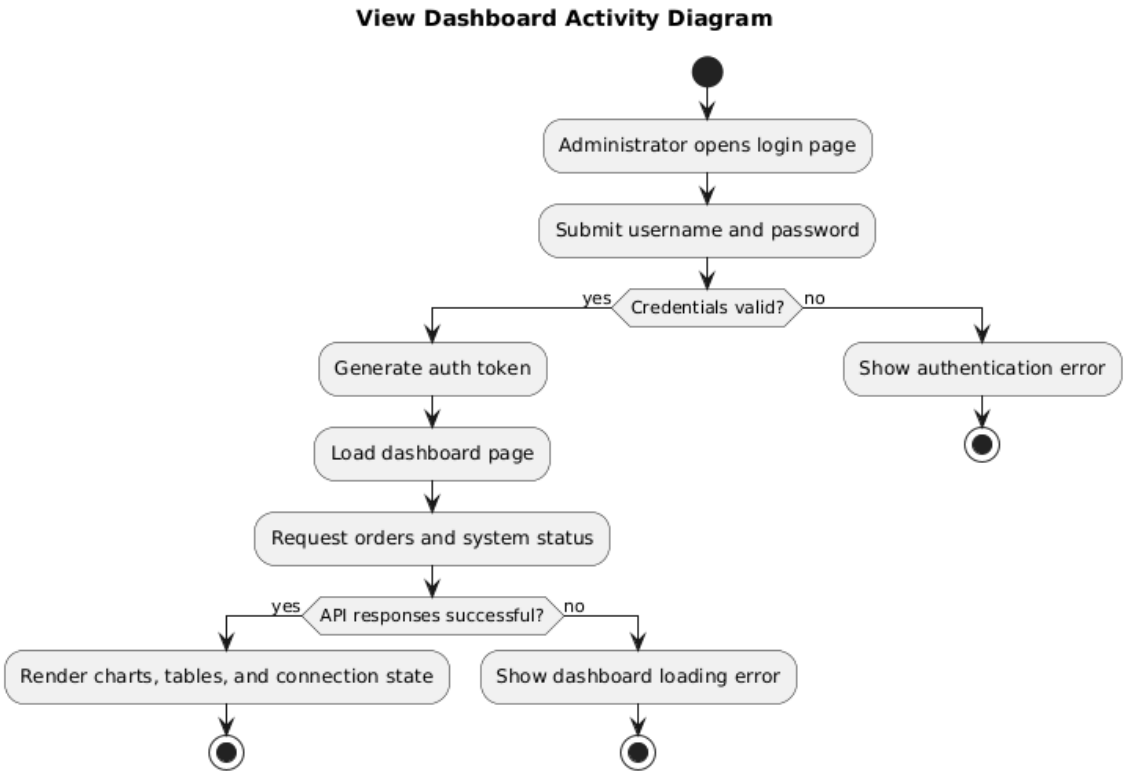
Receive Reminders Activity Diagram

This activity diagram illustrates the workflow for the receive reminders use case.

3.1.3 **View Dashboard**

Field	**Details**
Use case name	**View Dashboard**
Related Requirements	Requirement 7
Goal in Context	An admin views the dashboard to monitor orders, appointments, and system activity.
Preconditions	The admin is authenticated and has access to the system. The system has stored order and appointment data.
Successful End Condition	The dashboard is successfully displayed with relevant system data.
Failed End Condition	The dashboard fails to load due to missing data or system error, and an error message is displayed.
Primary Actors	Admin
Secondary Actors	Orders Database, Appointments Database
Trigger	The admin opens the dashboard interface.
Included Cases	Fetch System Data
Main Flow	*Step* | *Action***1** | The admin accesses the dashboard. **2** *include::Fetch System Data* | The system retrieves order and appointment data from the database. **3** | The system processes and organizes the data. **4** | The system displays the dashboard with summaries and information.
Extensions	*Step* | *Branching Action***2.1** | The system cannot retrieve one or more data sources.

	2.2 | The system displays partial data or an error message.

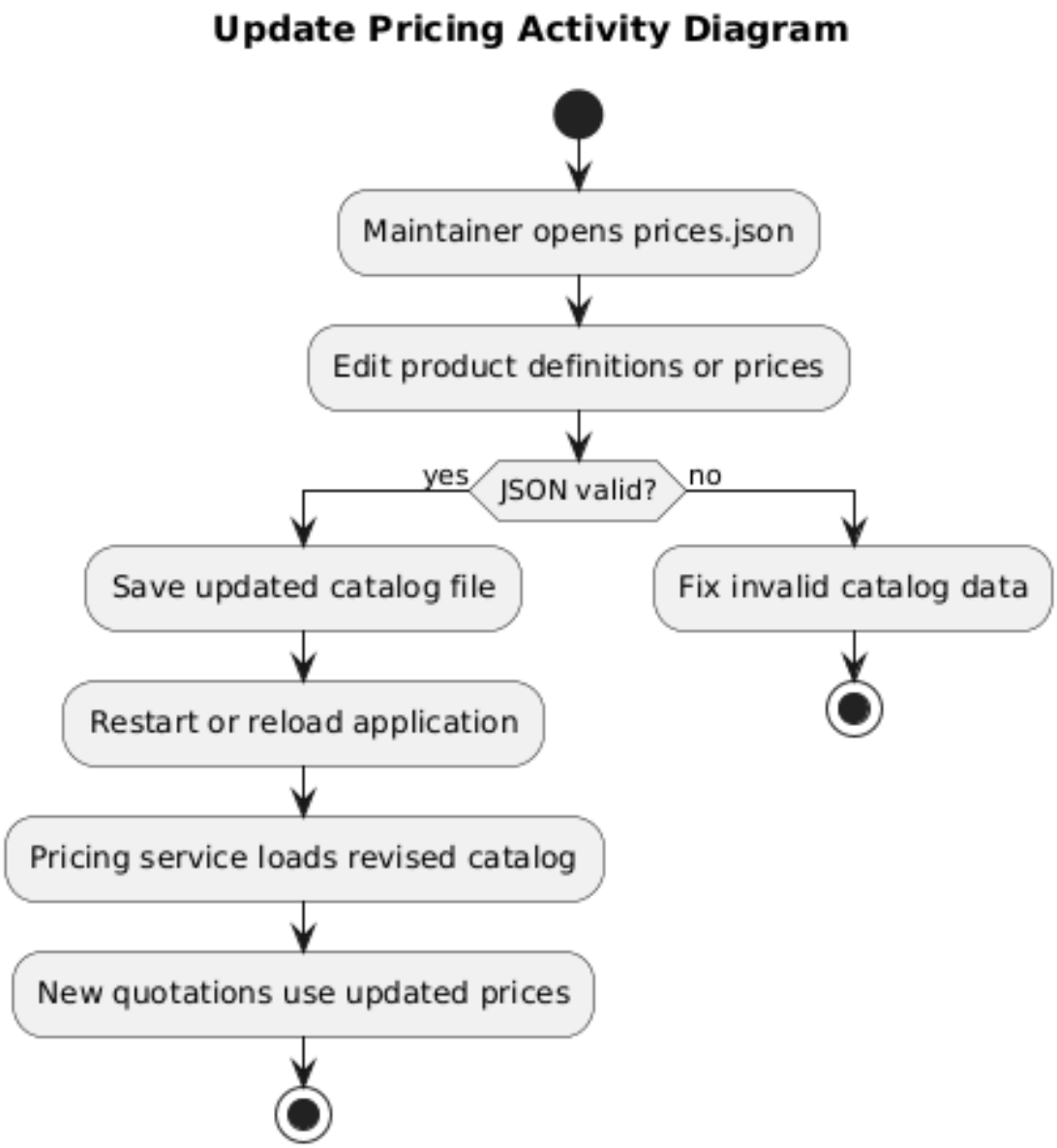


View Dashboard Activity Diagram

This activity diagram illustrates the workflow for the view dashboard use case.

3.1.4 **Update Pricing**

Field	**Values**
Use case name	**Update Pricing**
Related Requirements	Requirement 7
Goal in Context	An admin updates pricing information so that quotations and invoices reflect current prices.
Preconditions	The admin is authenticated and has access to pricing data. Pricing records exist in the system.
Successful End Condition	The pricing information is successfully updated and saved in the system.
Failed End Condition	The pricing update fails due to invalid input or system error, and an error message is displayed.
Primary Actors	Admin
Secondary Actors	Pricing Database
Trigger	The admin selects the option to update pricing.
Included Cases	Validate Pricing Input
Main Flow	StepAction1The admin accesses the pricing management section.2The system displays current pricing information.3The admin enters updated pricing values.4 include::Validate Pricing InputThe system validates the new pricing values.5The system saves the updated pricing information.6The system confirms the pricing update.
Extensions	StepBranching Action3.1The admin enters invalid or incomplete pricing values.3.2The system prompts the admin to correct the input.5.1The pricing fails to save due to a system error.5.2The system displays an error message and logs the issue.

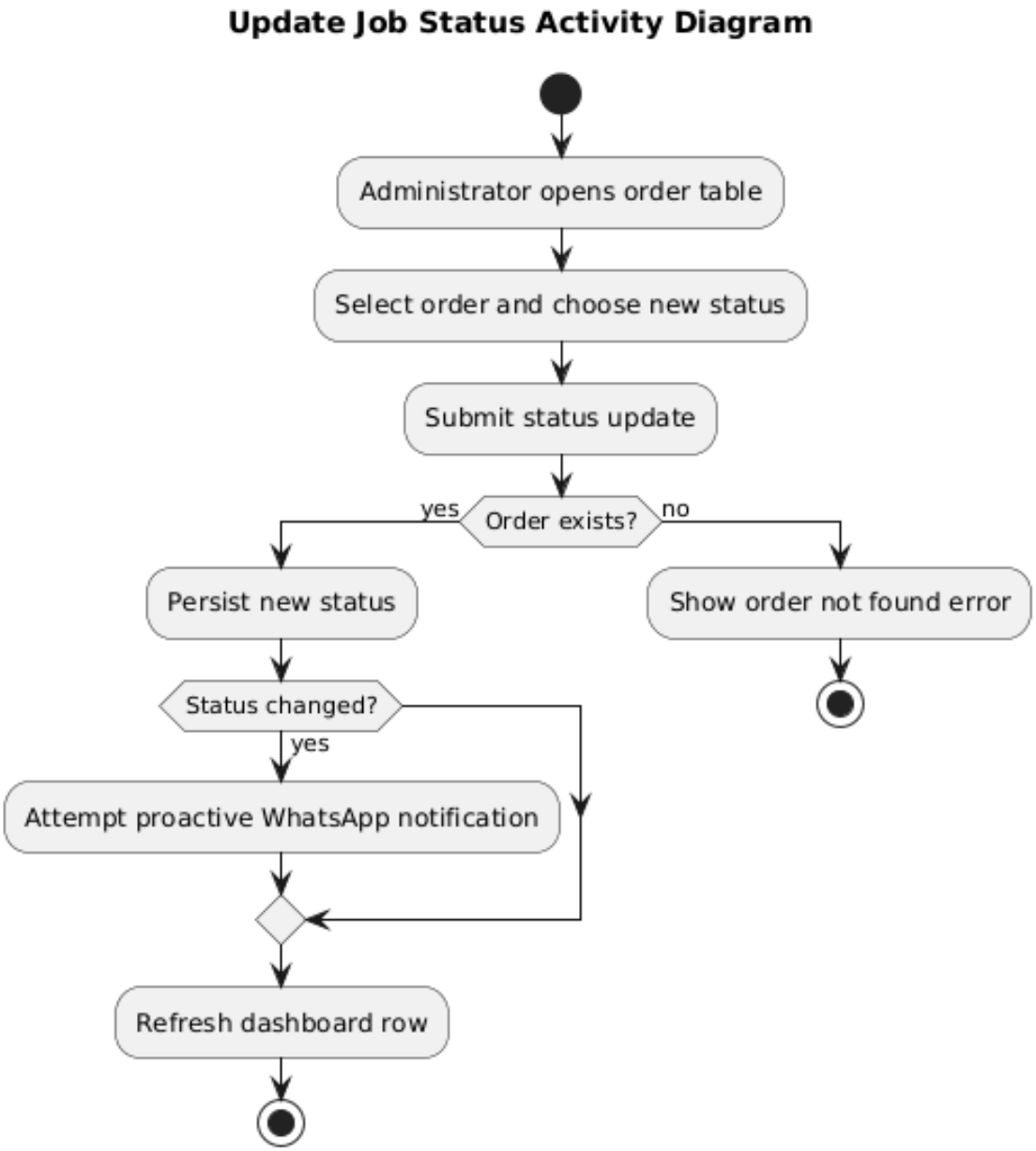


Update Pricing Activity Diagram

This activity diagram illustrates the workflow for the update pricing use case.

3.1.5 **Update Job Status**

Field	**Values**
Use case name	**Update Job Status**
Related Requirements	Requirement 5
Goal in Context	An admin updates the status of a job so that customers can receive accurate and up-to-date information.
Preconditions	The admin is authenticated and the job exists in the system.
Successful End Condition	The job status is successfully updated in the system.
Failed End Condition	The update fails because the job is not found or the status is invalid, and an error message is displayed.
Primary Actors	Admin
Secondary Actors	Orders Database
Trigger	The admin selects a job and chooses to update its status.
Included Cases	Fetch Job Status
Main Flow	StepAction1The admin selects a job from the dashboard.2 include::Fetch Job StatusThe system retrieves the current job details.3The admin selects a new job status.4The system validates the selected status.5The system updates the job status in the database.6The system confirms the update.
Extensions	StepBranching Action1.1The selected job does not exist.1.2The system displays an error message.4.1The selected status is invalid.4.2The system prompts the admin to select a valid status.



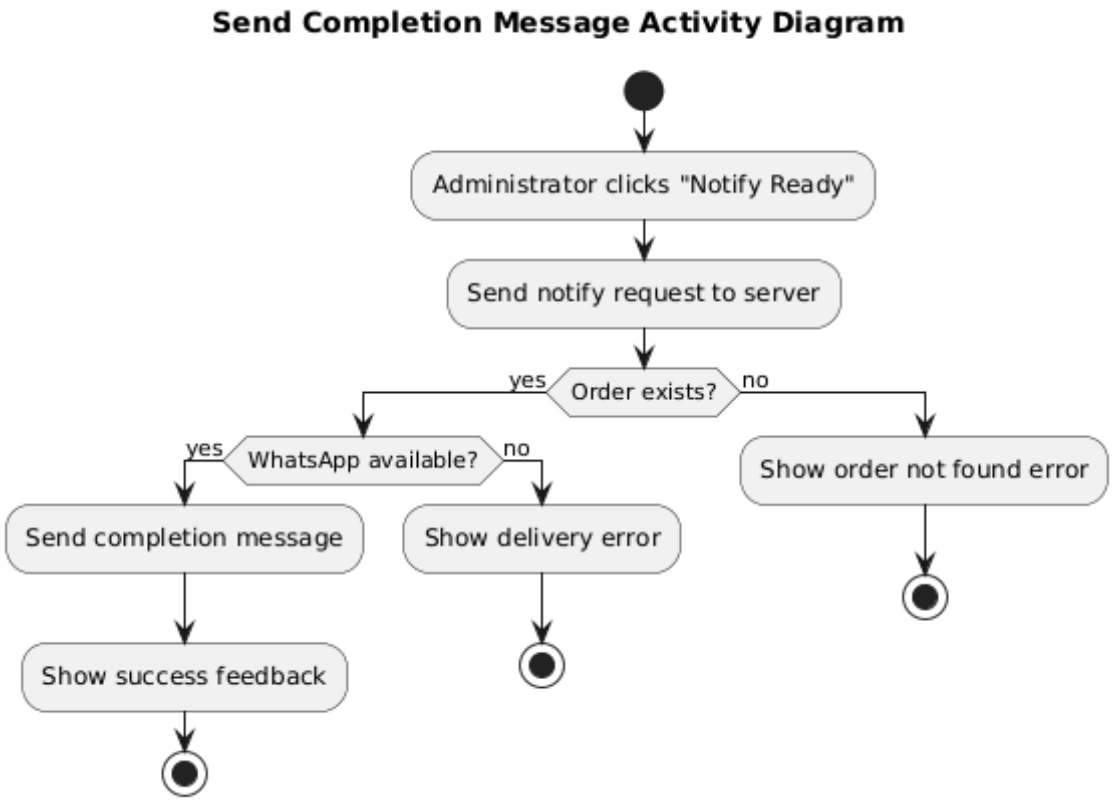
Update Job Status Activity Diagram

This activity diagram illustrates the workflow for the update job status use case.

3.1.6 **Send Completion Message**

Field	**Details**
Use case name	**Send Completion Message**
Related Requirements	Requirement 5.4
Goal in Context	A customer receives a final message via WhatsApp when their job is completed.
Preconditions	The job exists in the system and its status has been updated to "Completed". The customer has a valid WhatsApp number.
Successful End Condition	The completion message is successfully sent to the customer.
Failed End Condition	The message fails to send due to missing data or messaging errors, and the system logs the issue.
Primary Actors	WhatsApp API
Secondary Actors	Orders Database, Admin
Trigger	The job status is updated to "Completed".
Included Cases	Fetch Customer Details
Main Flow	**Step 1:** The system detects that the job status has changed to "Completed". **Step 2:** *include::Fetch Customer Details* The system retrieves the customer's contact information. **Step 3:** The system generates a completion message. **Step 4:** The system sends the message via WhatsApp. **Step 5:** The customer receives the completion message.

Extensions	**Step 2.1:** Customer contact information is missing or invalid. **Step 2.2:** The system logs the issue and stops the process. **Step 4.1:** The message fails to send due to API or network issues. **Step 4.2:** The system retries or logs the failure for admin review.



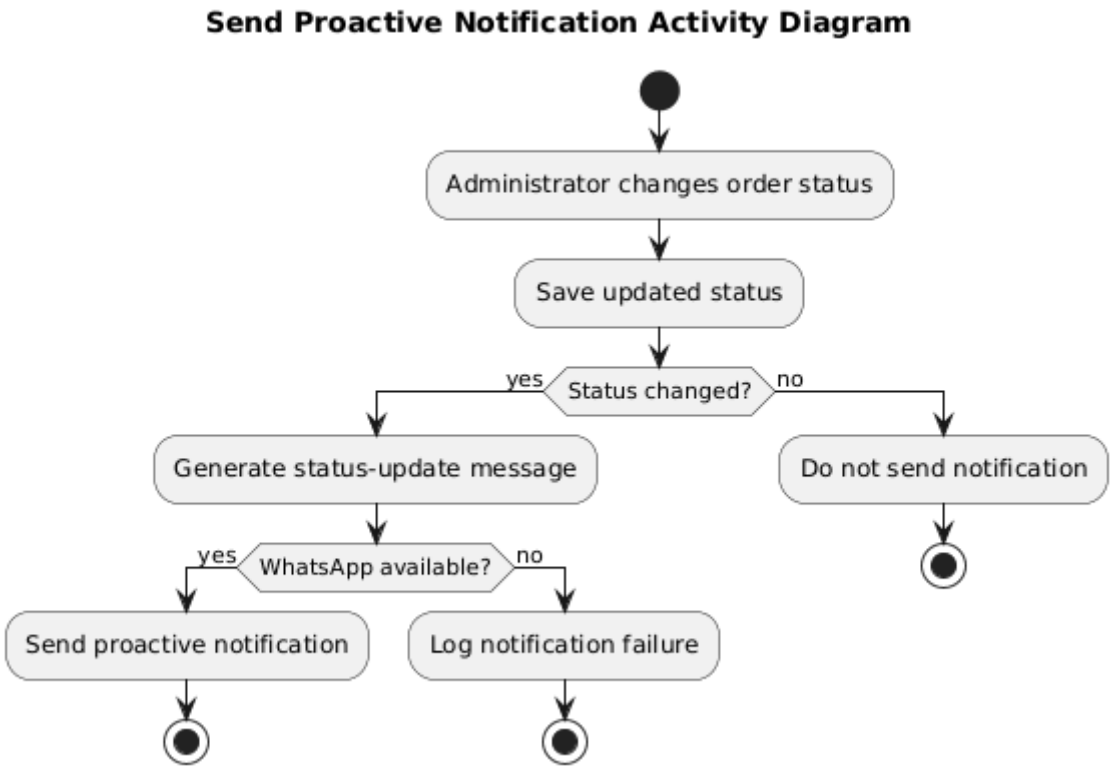
Send Completion Message Activity Diagram

This activity diagram illustrates the workflow for the send completion message use case.

3.1.7 **Send Proactive Notification**

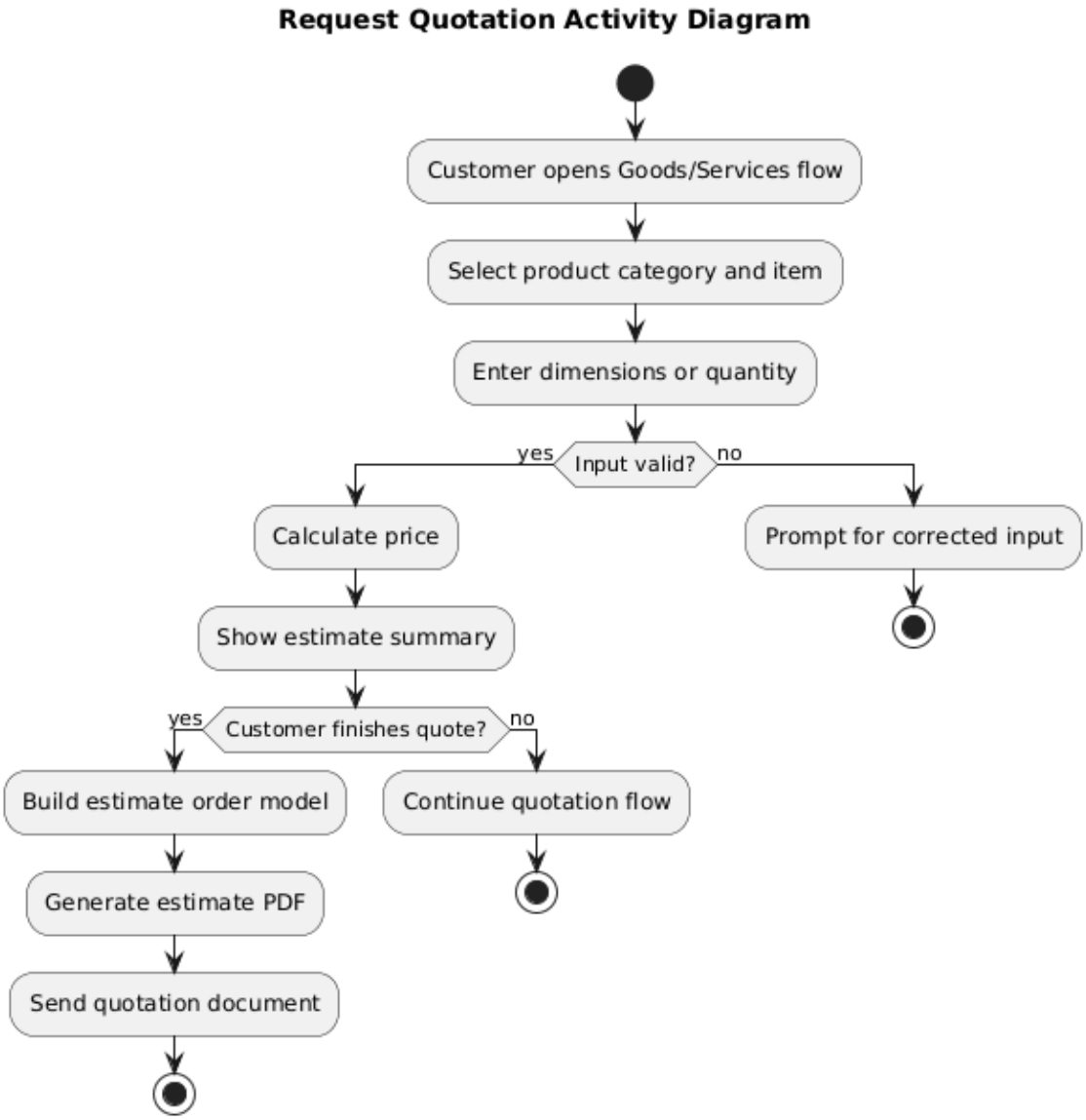
Field	**Details**
Use case name	**Send Proactive Notification**
Related Requirements	Requirement 5.3
Goal in Context	The system sends a notification to a customer to provide updates about their job status or important information without the customer initiating the request.
Preconditions	The customer exists in the system and has a valid WhatsApp number. A triggering event (e.g., status change or update) has occurred.
Successful End Condition	The notification is successfully sent to the customer.
Failed End Condition	The notification fails to send due to missing data or messaging errors, and the system logs the issue.
Primary Actors	System
Secondary Actors	Orders Database, WhatsApp API, Admin
Trigger	A system event occurs or an admin initiates a proactive notification.
Included Cases	Fetch Customer Details
Main Flow	*Step* **Step 1:** The system detects a triggering event or admin action. **Step 2:** *include::Fetch Customer Details* The system retrieves the **Step 3:** The system generates a notification message.

	Step 4: The system sends the notification via WhatsApp. **Step 5:** The customer receives the notification.
Extensions	**Step 2.1:** Customer contact information is missing or invalid. **Step 2.2:** The system logs the issue and stops the process. **Step 4.1:** The message fails to send due to API or network issues. **Step 4.2:** The system retries or logs the failure for admin review.



Send Proactive Notification Activity Diagram

This activity diagram illustrates the workflow for the send proactive notification use case.



Request Quotation Activity Diagram

This activity diagram illustrates the workflow for the request quotation use case.

3.1.8 User Interface

No separate activity diagram is included here because this interaction is already covered by the chatbot and dashboard use-case flows.

Field	Details
Use	**User Interface**

case name	
Related Requirements	Requirement 1
Goal in Context	This allows the customer to interact with the system easily.
Preconditions	WhatsApp API, Stable Internet Connection, Active Server
Successful End Condition	The user can successfully select a valid option by entering a number or keyword.
Failed End Condition	An automated help message prompts the user to re-enter their selection.
Primary Actors	Customer
Secondary Actors	None
Trigger	The customer sends a message to the business WhatsApp number.
Included Cases	None
Main Flow	*Step* ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ *Action* **Step 1:** The customer sends a message to the business WhatsApp number.

	Step 2: The system responds with a GUI-like structured message listing available service options. **Step 3:** The customer replies with a valid number or keyword corresponding to a service. **Step 4:** The system validates the input and confirms the selected option. **Step 5:** The system provides a response or next-step instruction.
Extensions	*Step ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ÃƒÆ'Ã‚Â¢ÃƒÂ¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€ Ã¢â‚¬â„¢ Branching Action* **Step 3.1:** The customer enters an invalid option. **Step 3.2:** An error message is displayed to the customer.

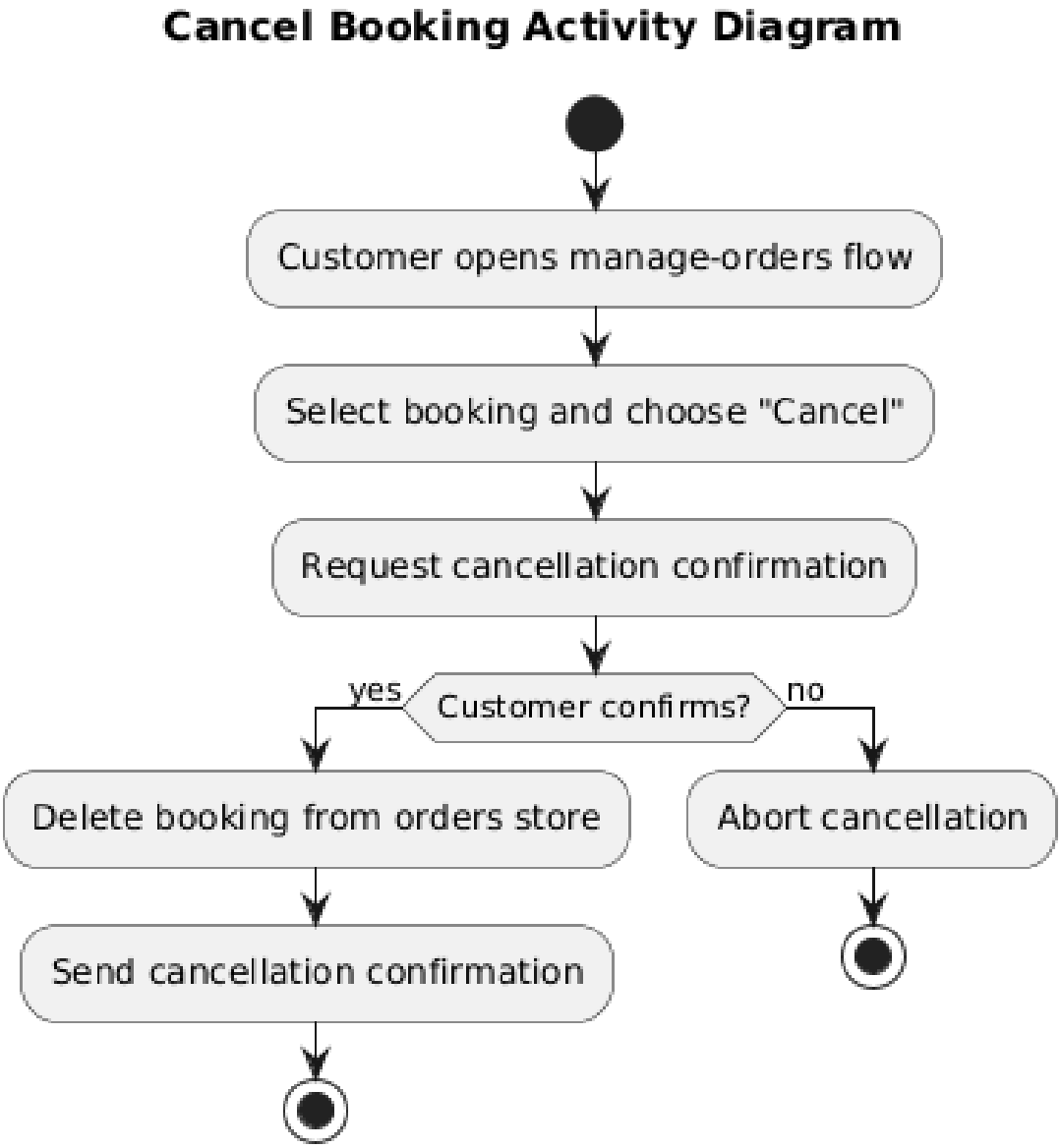
3.1.9 **Receive Report**

No separate activity diagram is included here because report delivery is covered by the implemented quotation and invoice document flows.

Field	**Details**
Use case name	**Receive Report**
Related Requirements	Requirement 3
Goal in Context	A customer needs to receive a detailed report of their order(s).
Preconditions	The customer made at least one order.
Successful End Condition	A report is created and sent to the customerÃƒÆ'Ã‚¢Ãƒâ€Ã‚¢â‚¬Å¡Ã‚¬Ãƒâ€Ã‚¢â‚¬Å¾Ã‚¢s email.
Failed End Condition	The report is not sent.
Primary Actors	Customer
Secondary Actors	Orders Database
Trigger	Confirmation to the system from the customer to create a report.
Included Cases	Generate Report
Main Flow	**Step 1:** The customer confirms that the report should be sent. **Step 2:** The system retrieves from Orders Database all details of orders associated with the customer. **Step 3:** *include::Generate Report* The system generates the report from the details. **Step 4:** The report is saved to Orders Database.

	Step 5: The report is sent to the customerÃƒÆ’Ã‚Â¢Ãƒâ€Â¬Å¡Ã‚Â¬ÃƒÂ¢Ãâ€Â¬Å¾Ã‚Â¢s email.
Extensions	**Step 3.1:** The report is not generated. **Step 3.2:** An error message is displayed to the customer.

3.1.10 **Edit Booking**

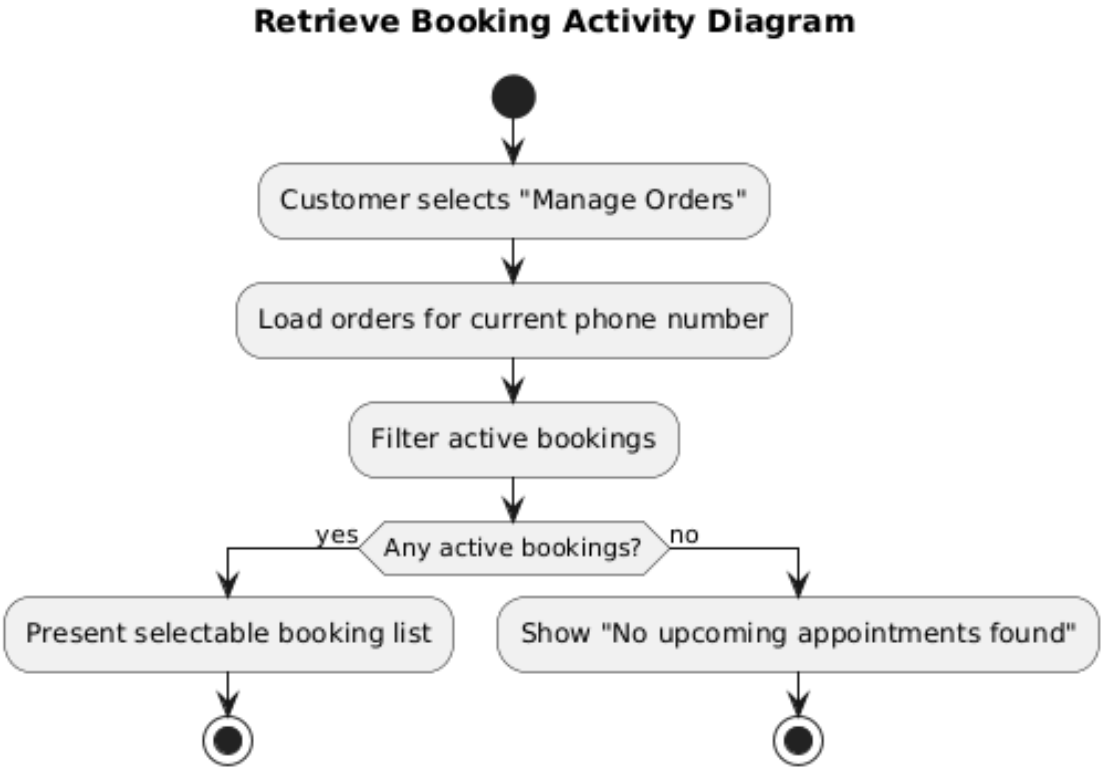


Cancel Booking Activity Diagram
This activity diagram illustrates the workflow for the cancel booking use case.

Field	**Details**
Use case name	**Edit Booking**

Related Requirements	Requirement 5, Requirement 7
Goal in Context	A customer needs to change the details of a booking.
Preconditions	The customer made a booking.
Successful End Condition	The details of the booking are changed.
Failed End Condition	The booking is unchanged.
Primary Actors	Customer
Secondary Actors	Orders Database
Trigger	The customer requests the system to allow them to change a bookingÃƒÆ’Ã‚Â¢Ãƒâ€Â¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€Â¢Ã¢â‚¬Å¾Ã‚Â¢s details.
Included Cases	Retrieve Booking
Main Flow	**Step 1:** The customer requests the system to allow a change to bookingÃƒÆ’Ã‚Â¢Ãƒâ€Â¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€Â¢Ã¢â‚¬Å¾Ã‚Â¢s details. **Step 2:** The customer selects a booking. **Step 3:** *include::Retrieve Booking* The system retrieves the bookingÃƒÆ’Ã‚Â¢Ãƒâ€Â¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€Â¢Ã¢â‚¬Å¾Ã‚Â¢s details from Orders Database. **Step 4:** The customer changes the bookingÃƒÆ’Ã‚Â¢Ãƒâ€Â¢Ã¢â‚¬Å¡Ã‚Â¬Ãƒâ€Â¢Ã¢â‚¬Å¾Ã‚Â¢s details. **Step 5:** The booking is updated in Orders Database.
Extensions	**Step 3.1:** The booking is not found.

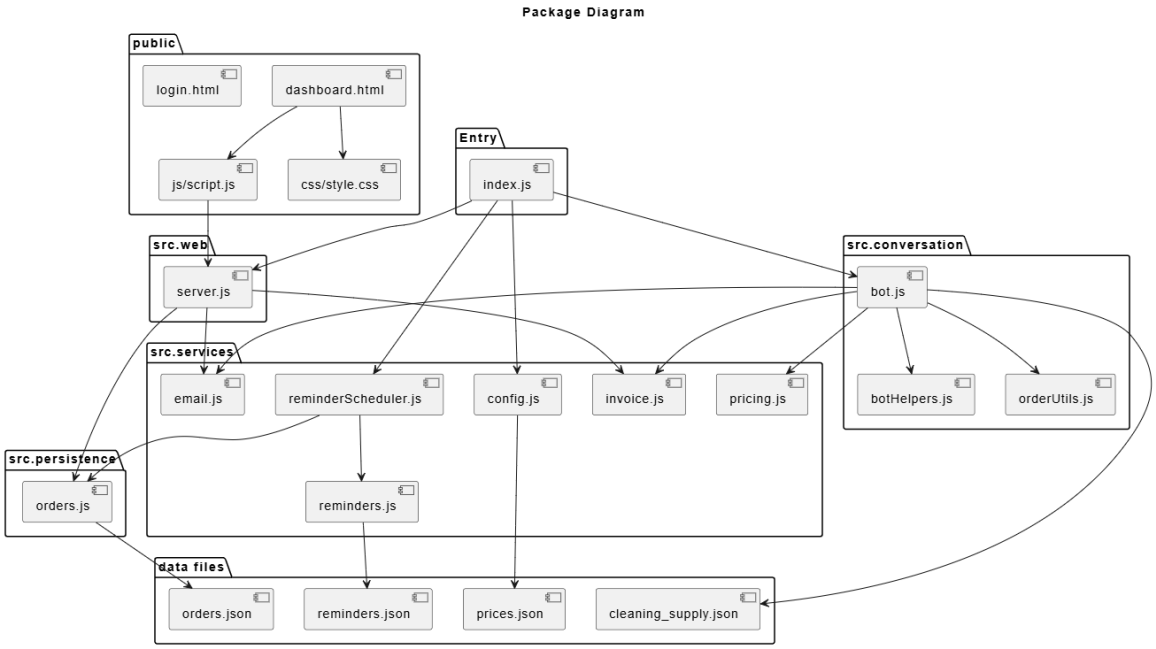
	Step 3.2: An error message is displayed. **Step 5.1:** The booking is not updated. **Step 5.2:** The booking remains the same in Orders Database.



Retrieve Booking Activity Diagram
This activity diagram illustrates the workflow for the retrieve booking use case.

4 Architectural Design

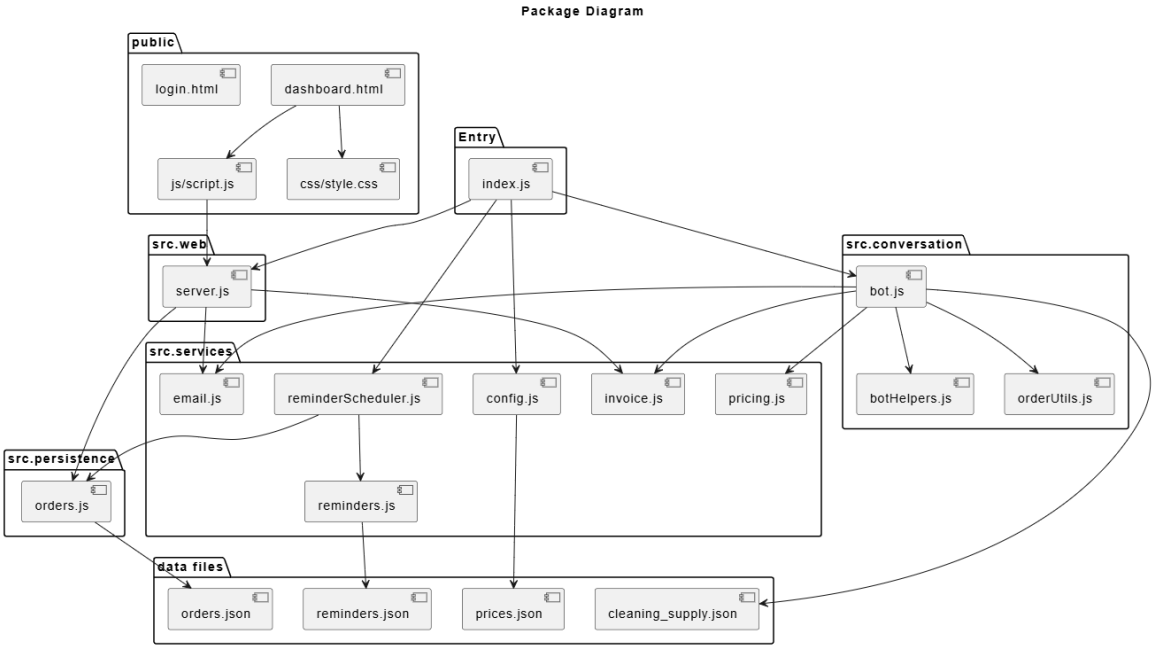
4.1 Package Diagram



Package Diagram

Shows how the project is grouped into major code and asset packages.

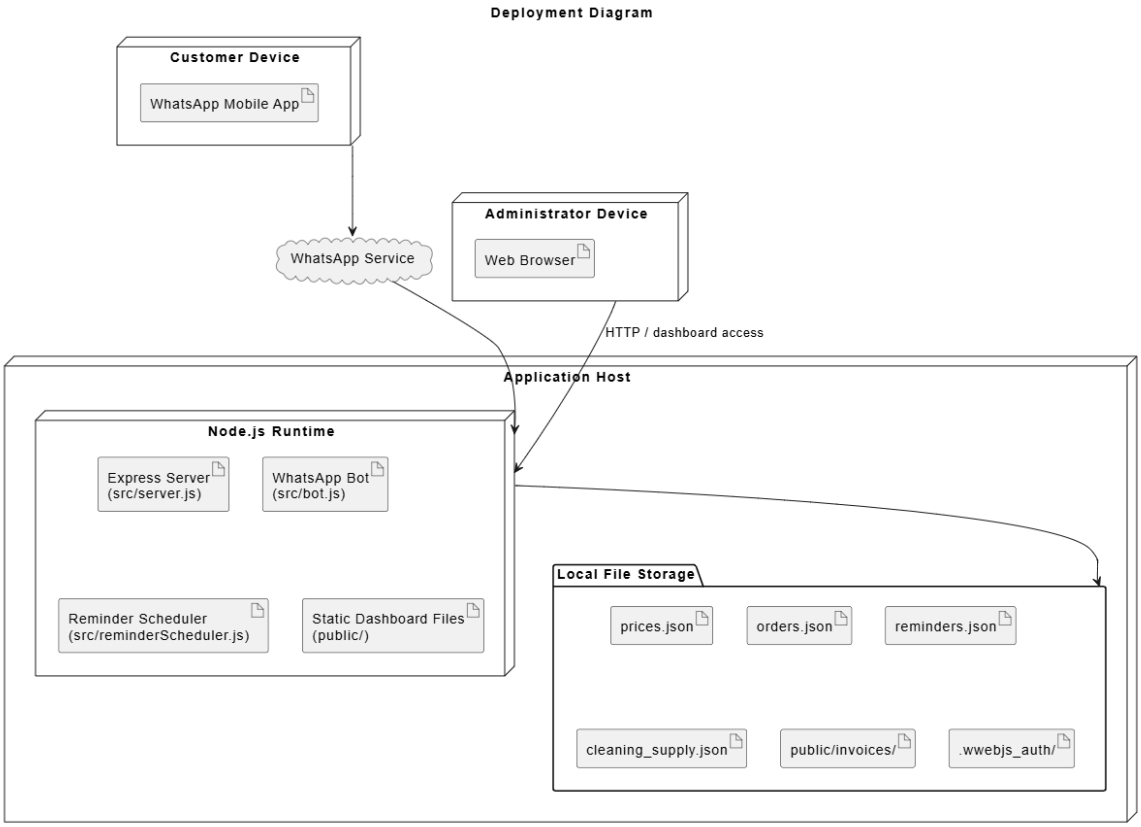
4.2 Component Diagram



Component Diagram

Shows the main runtime components and how they communicate.

4.3 Deployment Diagram

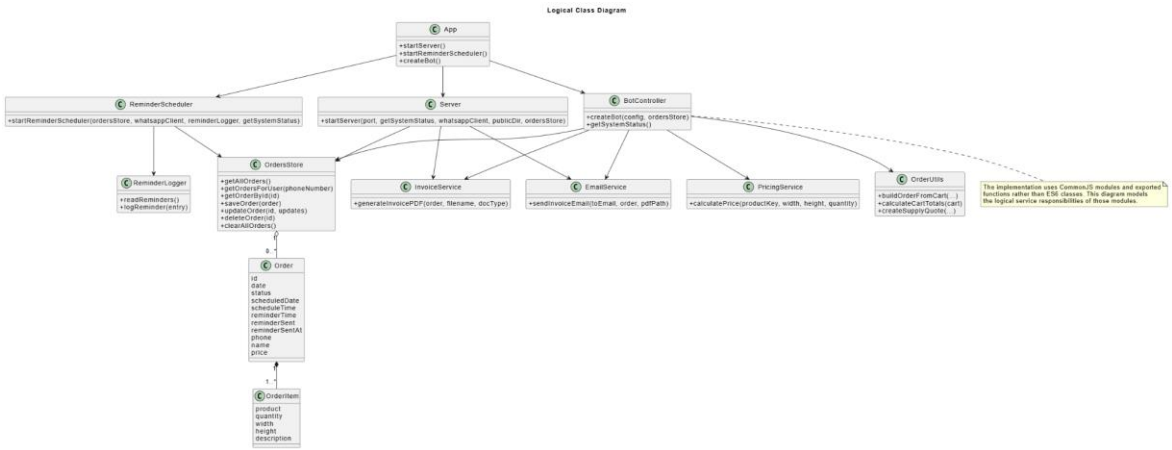


Deployment Diagram

Shows how the chatbot, dashboard, and storage are deployed in practice.

5 Detailed Design

5.1 Class Diagram

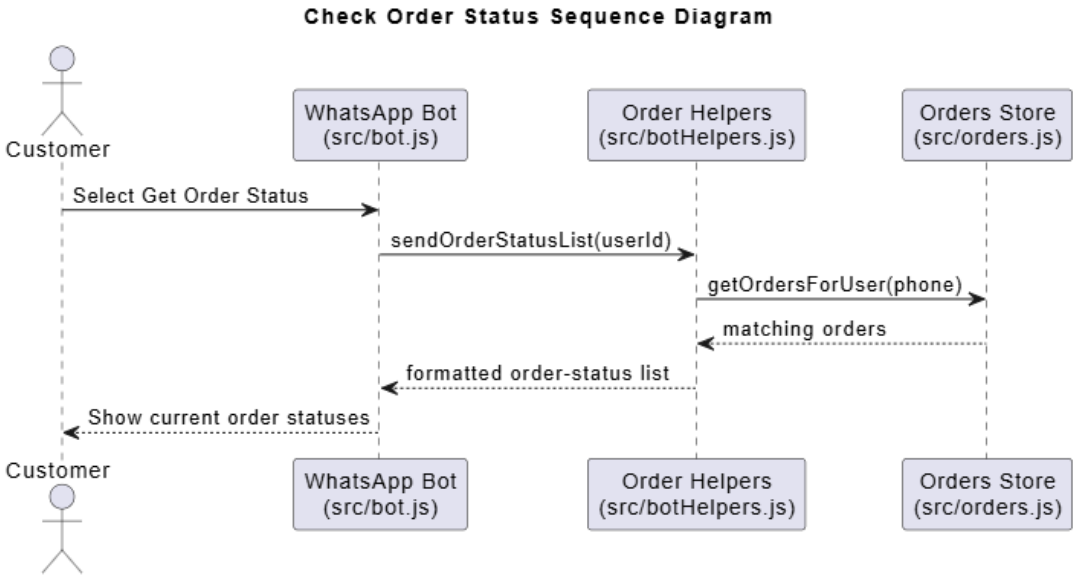


Class Diagram

Shows the main logical classes and responsibilities in the application.

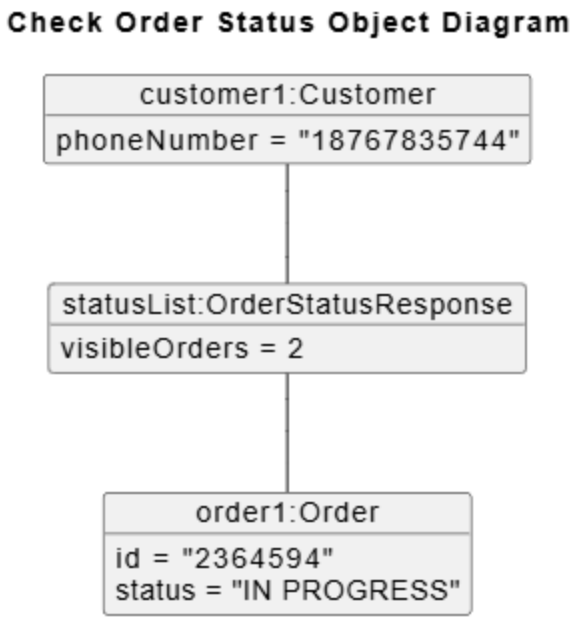
5.2 Sequence, Object & State Diagrams

5.2.1 Check Order Status



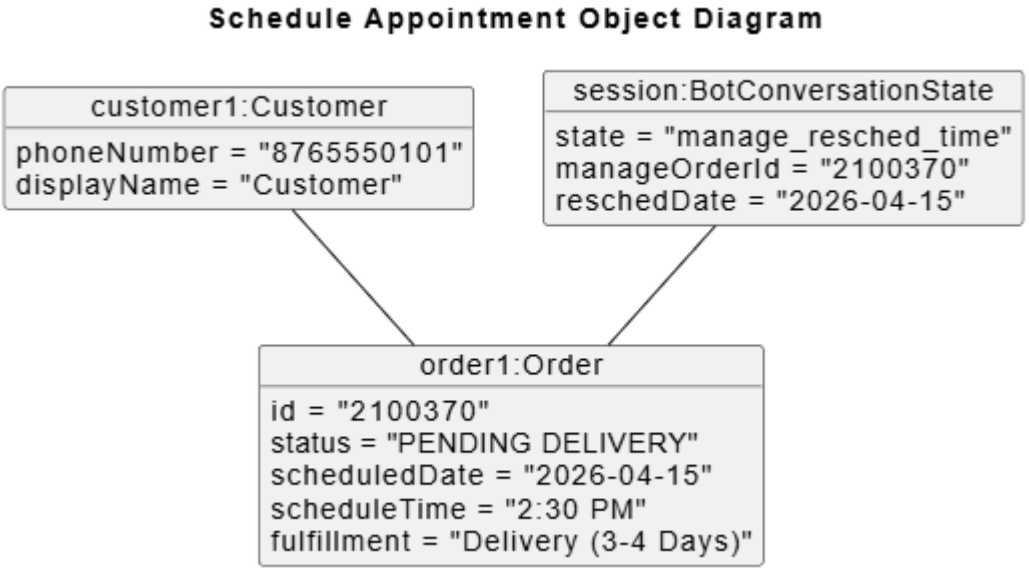
Check Order Status Sequence Diagram

Shows the interaction steps for retrieving order status through the bot.

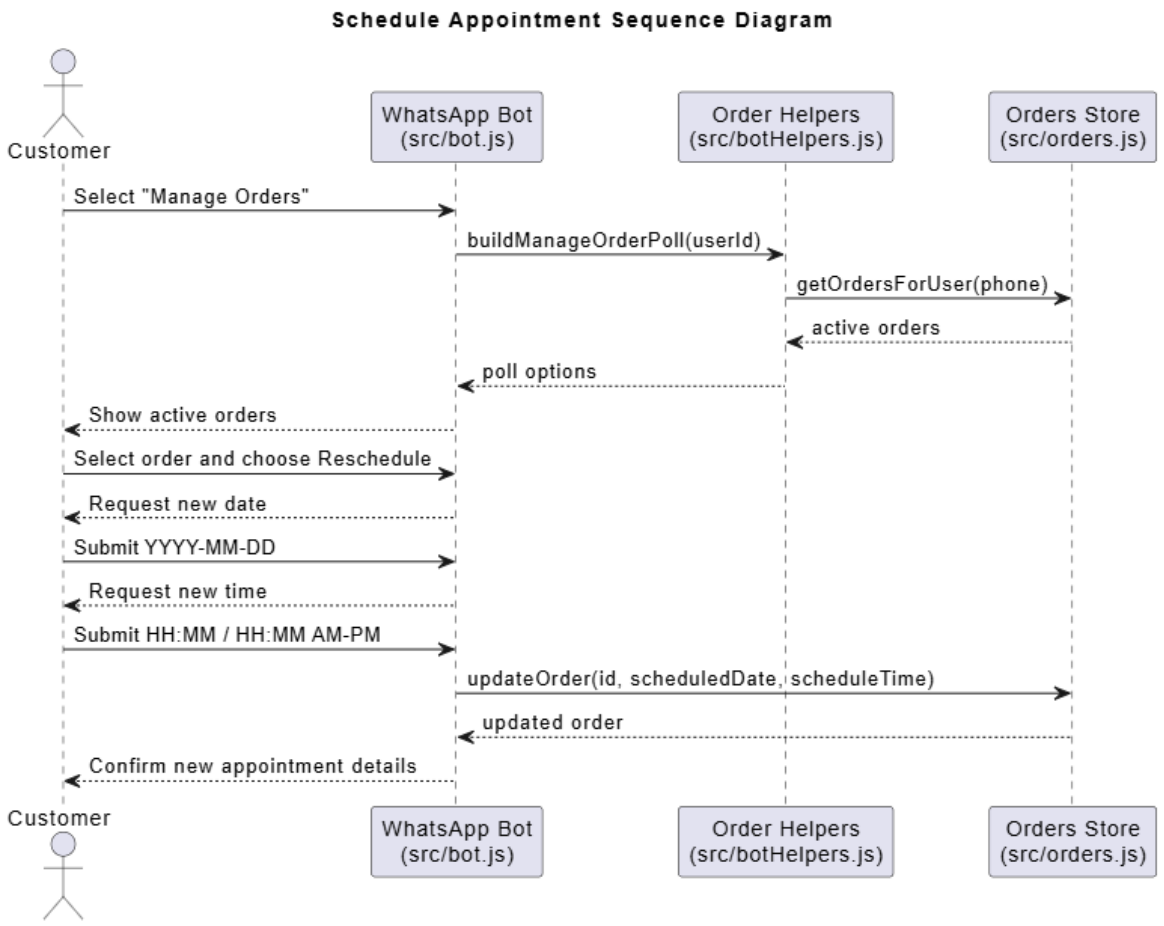


Check Order Status Object Diagram
Shows a runtime snapshot of the customer and order status response.

5.2.2 Schedule Appointment

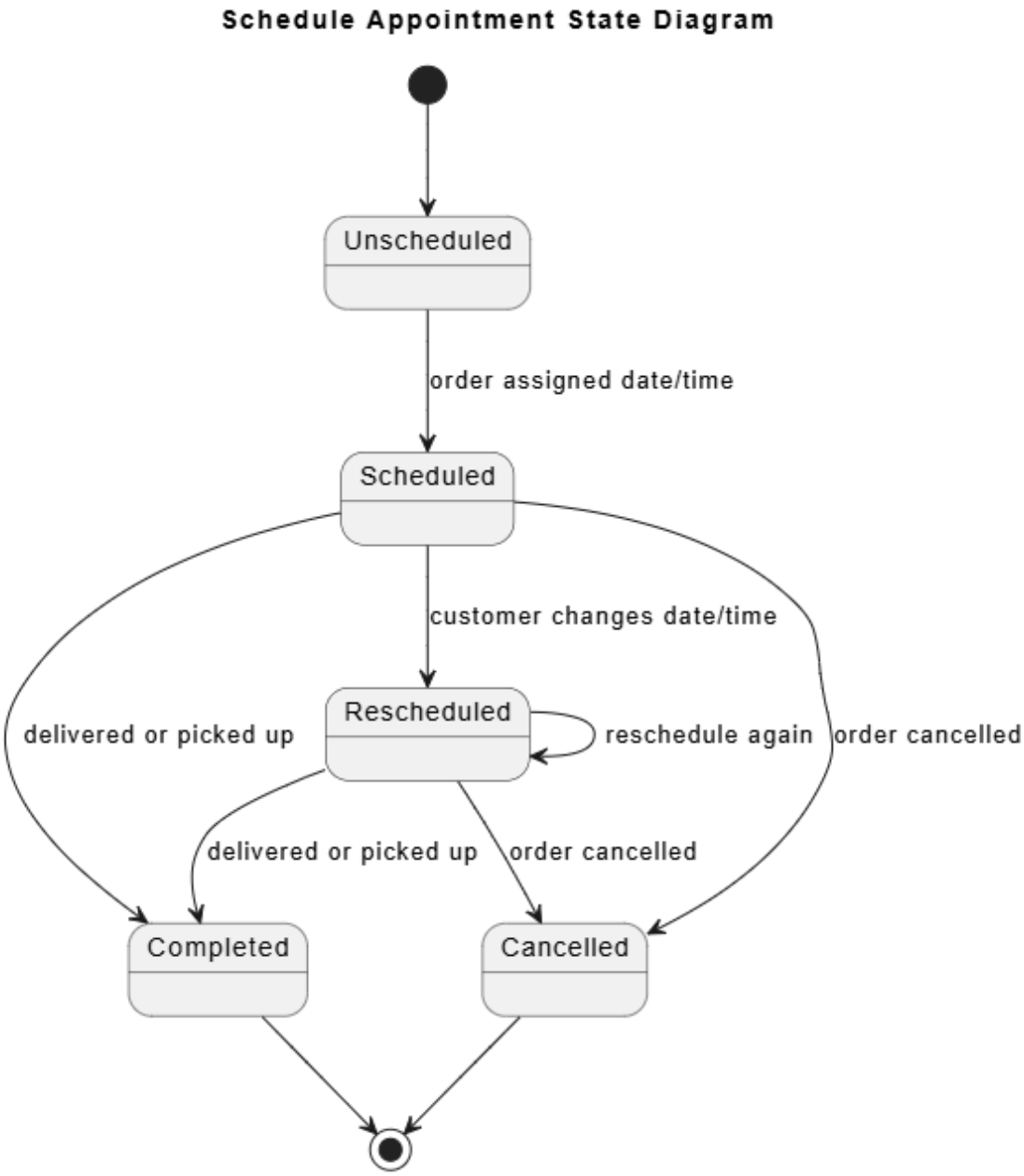


Schedule Appointment Sequence Diagram
Shows the interaction steps for rescheduling an existing appointment.



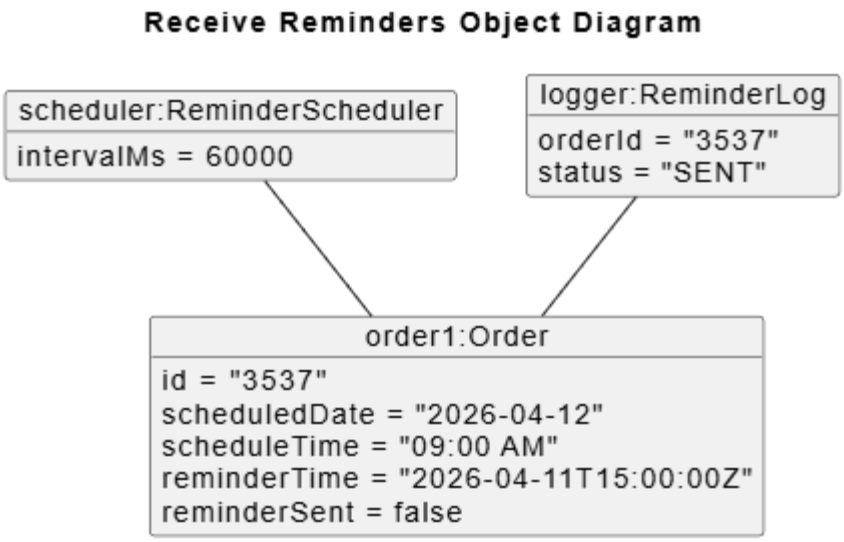
Schedule Appointment Object Diagram

Shows a runtime snapshot of the appointment rescheduling flow.



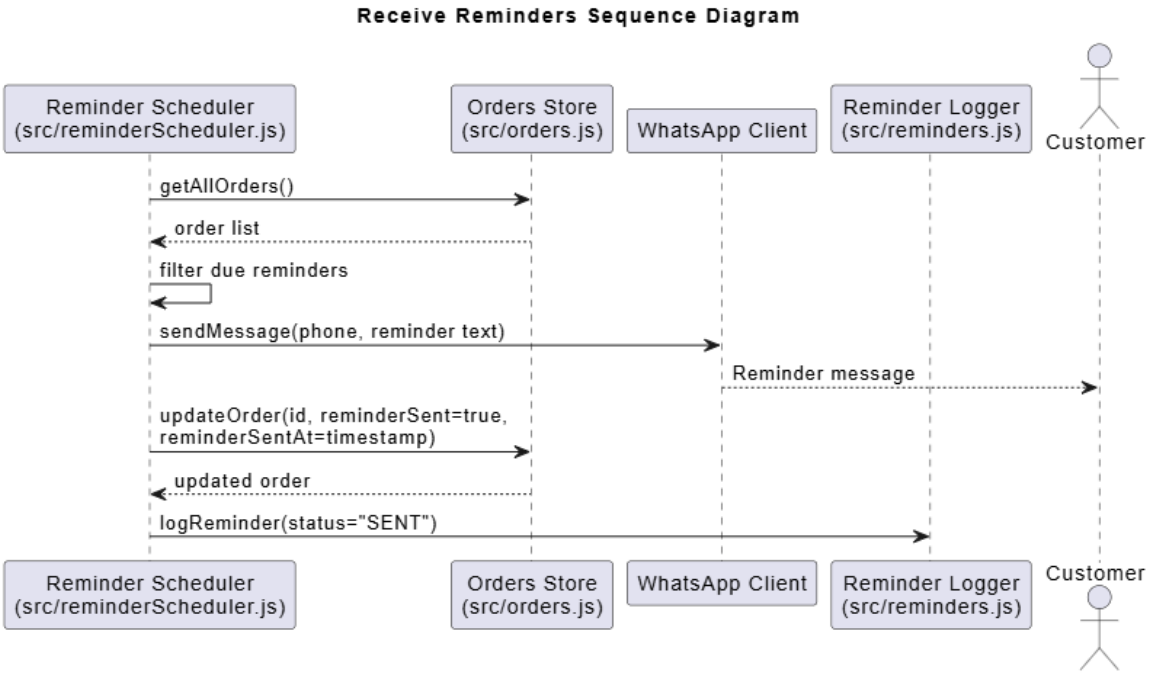
Schedule Appointment State Diagram
Shows the key states an appointment can move through in the system.

5.2.3 Receive Reminders



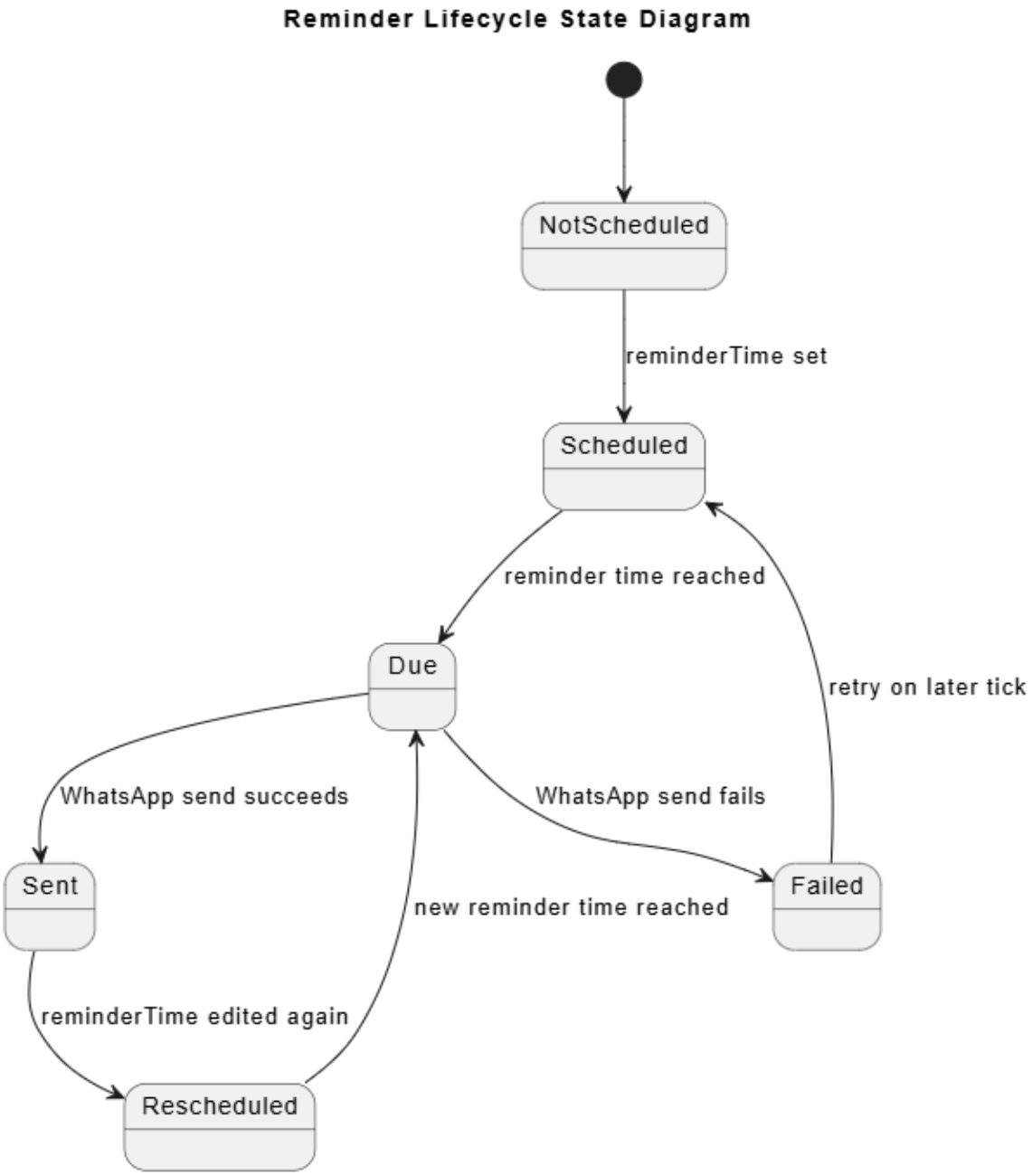
Receive Reminders Sequence Diagram

Shows how reminder checks, delivery, and logging occur.



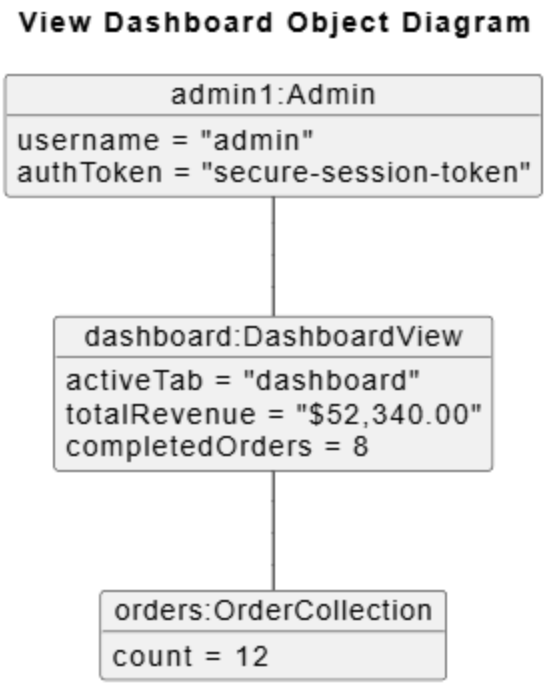
Receive Reminders Object Diagram

Shows a runtime snapshot of a due reminder and related objects.



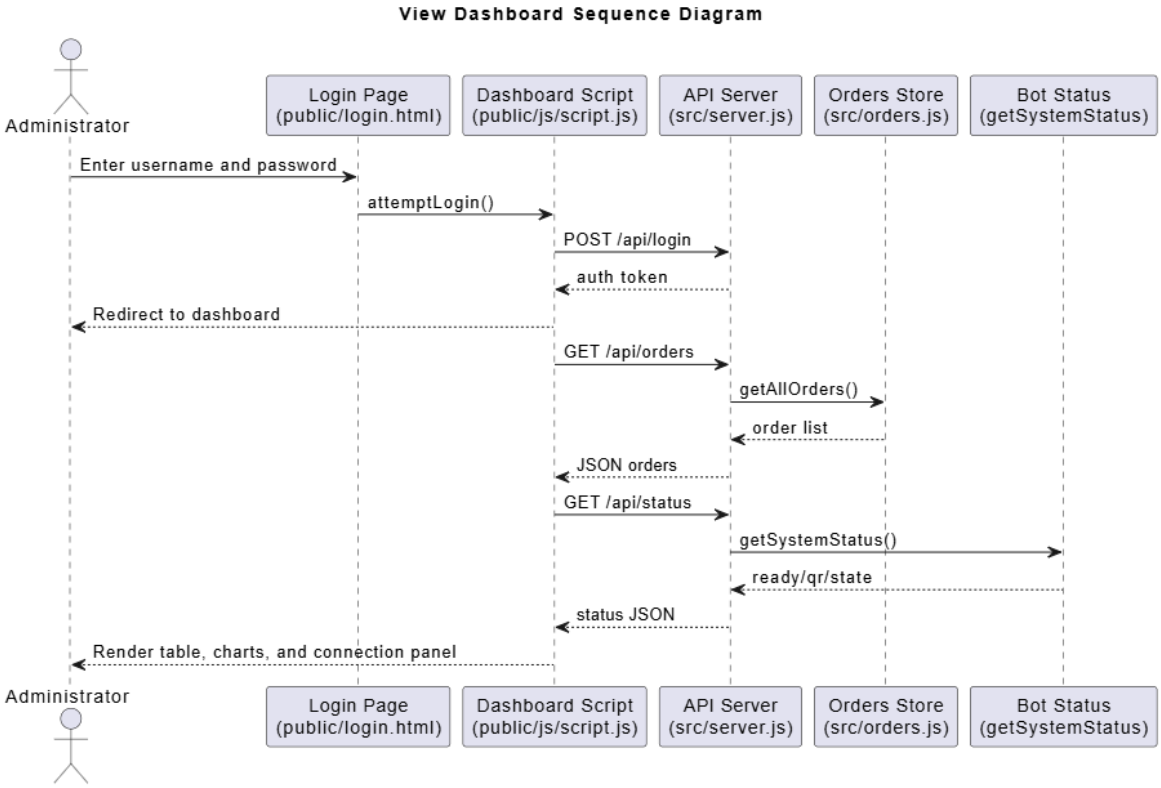
Receive Reminders State Diagram

Shows the reminder lifecycle from pending to sent or failed.



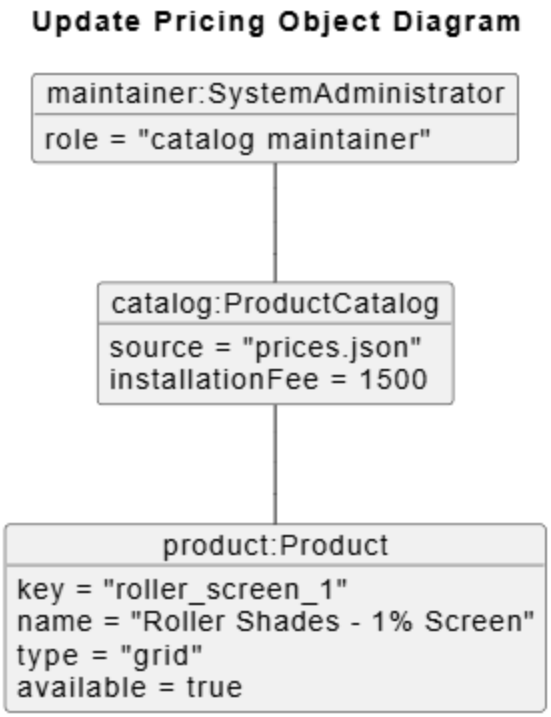
View Dashboard Sequence Diagram

Shows the login and dashboard loading interaction for an administrator.



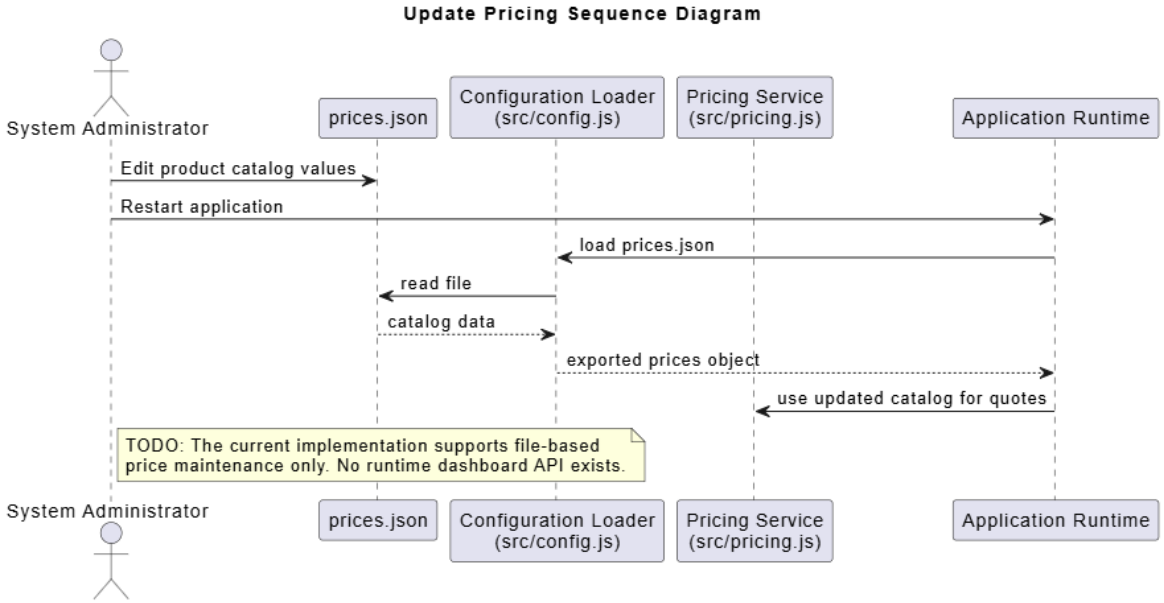
View Dashboard Object Diagram

Shows a runtime snapshot of the admin dashboard context.



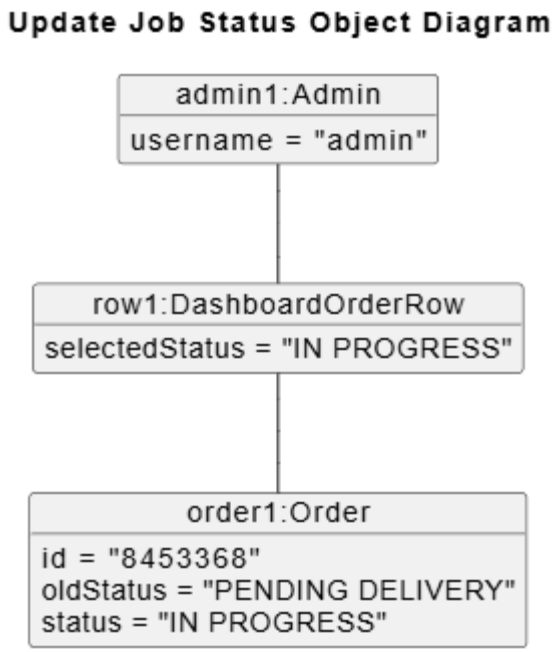
Update Pricing Sequence Diagram

Shows the file-based process used to update product pricing.



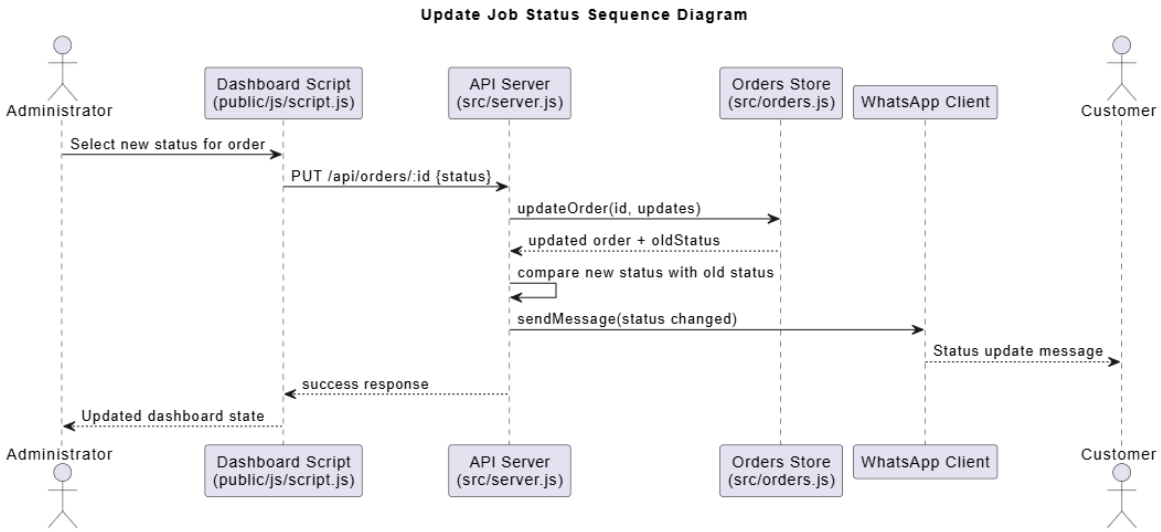
Update Pricing Object Diagram

Shows a runtime snapshot of the pricing catalog and maintainer context.



Update Job Status Sequence Diagram

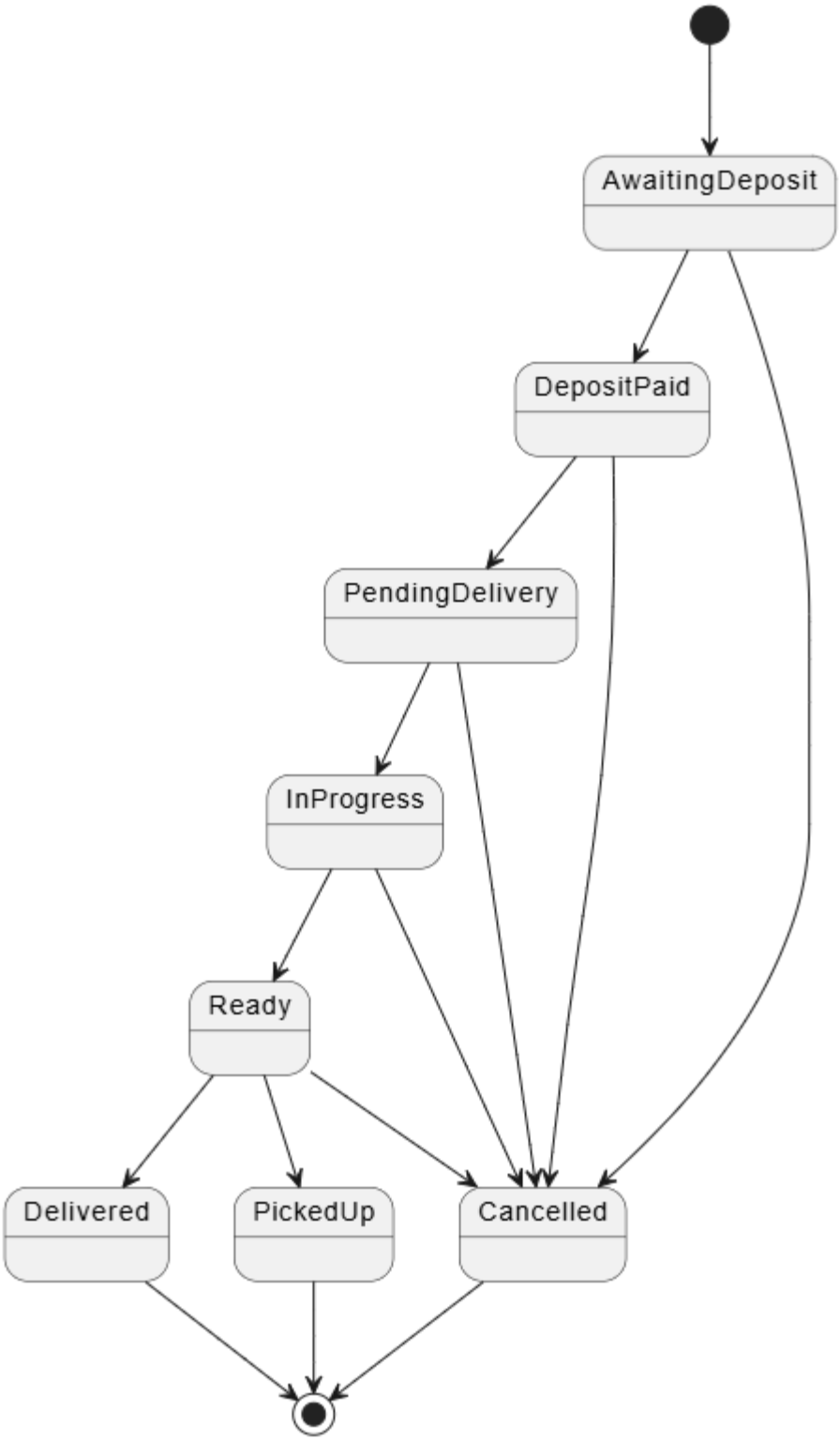
Shows how an administrator updates a job status from the dashboard.



Update Job Status Object Diagram

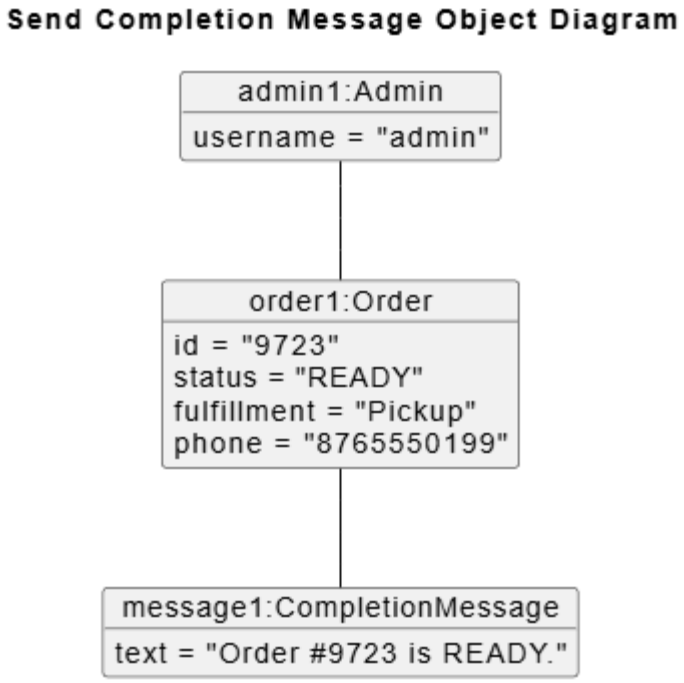
Shows a runtime snapshot of an order being updated by an administrator.

Order Status State Diagram



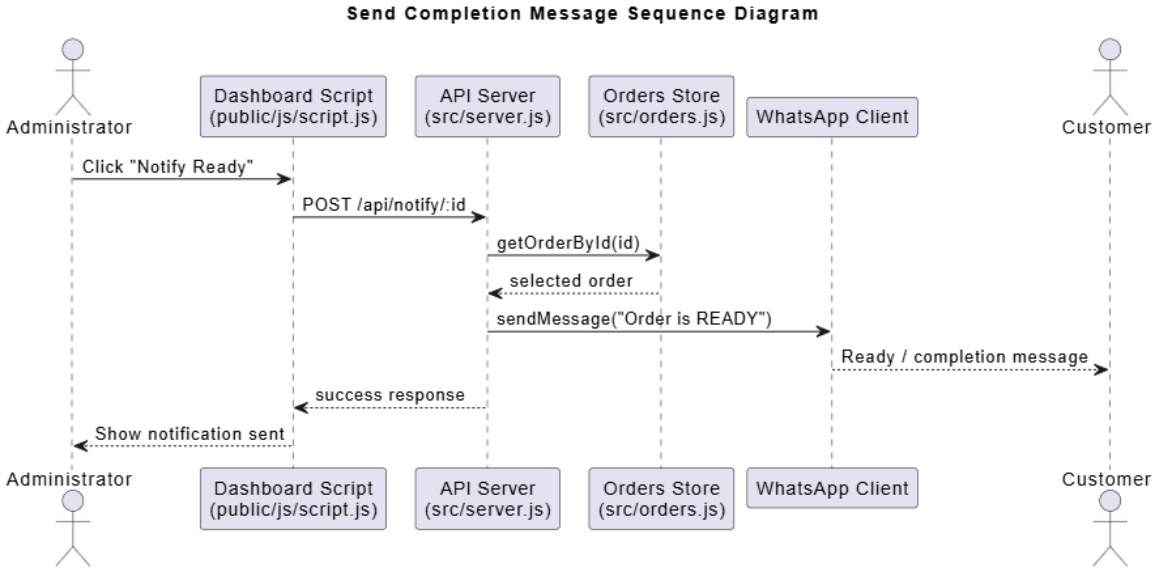
Update Job Status State Diagram

Shows the order status lifecycle used by the dashboard and notifications.



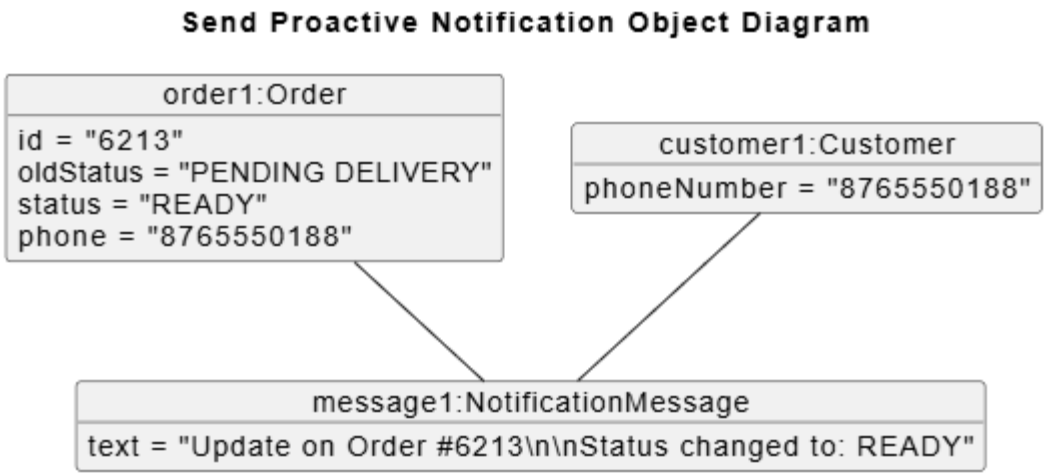
Send Completion Message Sequence Diagram

Shows the steps for sending a completion message to a customer.



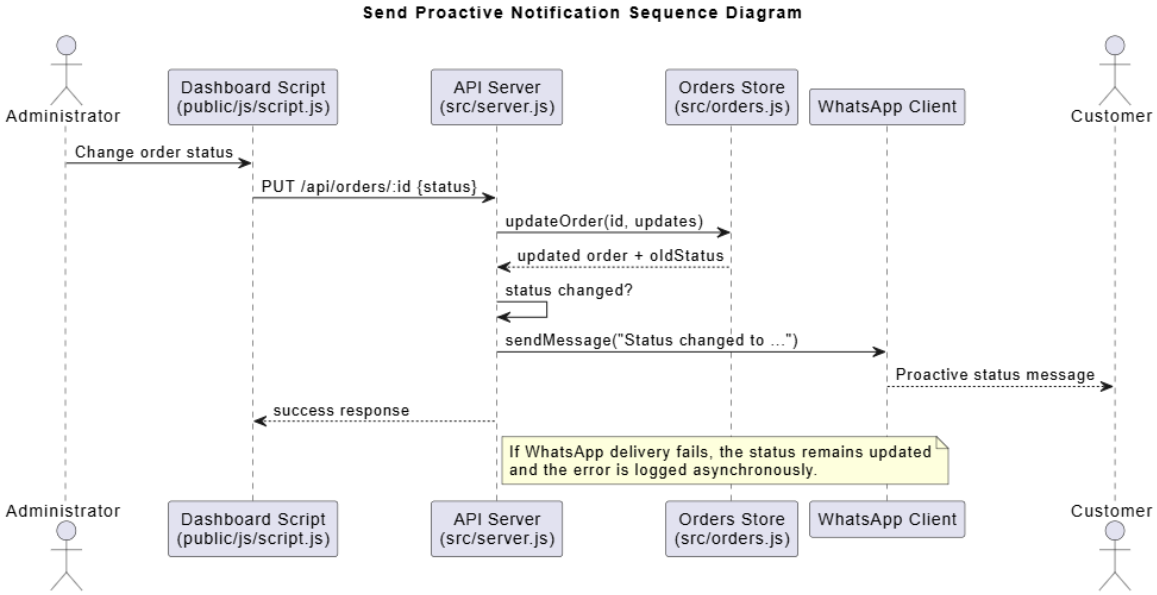
Send Completion Message Object Diagram

Shows a runtime snapshot of a completion notification event.



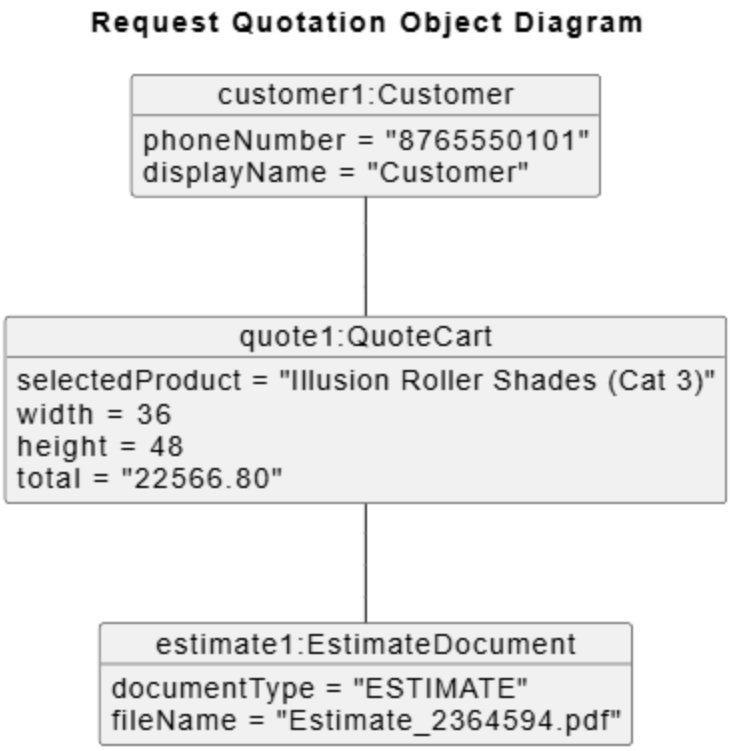
Send Proactive Notification Sequence Diagram

Shows the steps for sending an automatic proactive status update.



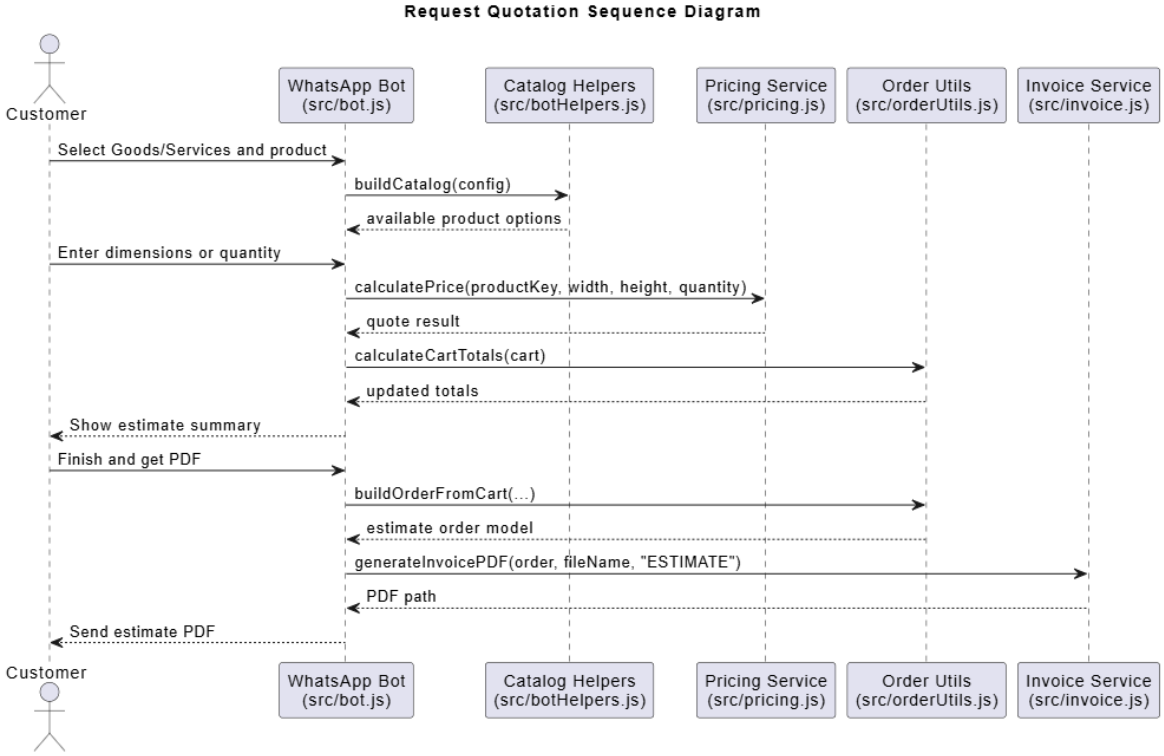
Send Proactive Notification Object Diagram

Shows a runtime snapshot of a proactive notification after a status change.



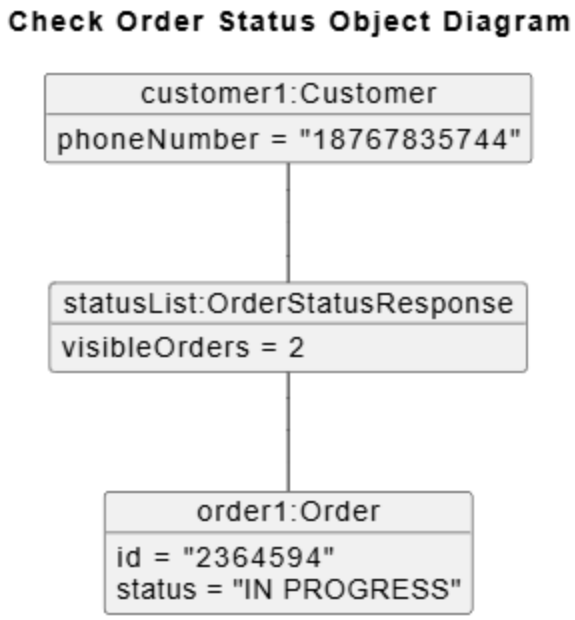
Request Quotation Sequence Diagram

Shows how the system gathers inputs and generates a quotation.



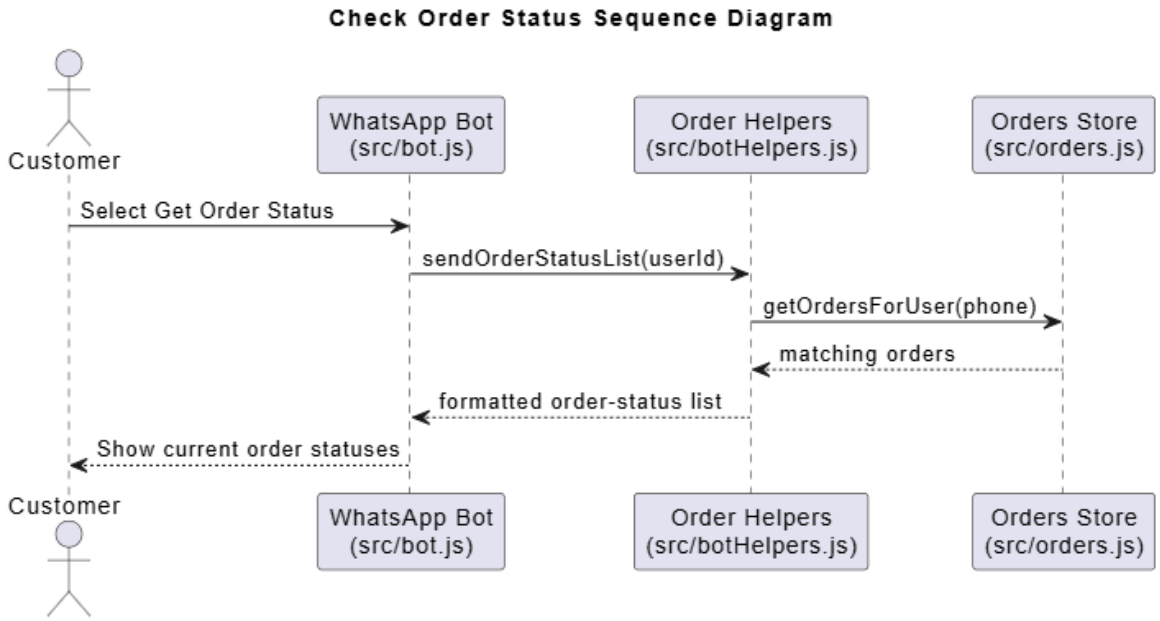
Request Quotation Object Diagram

Shows a runtime snapshot of the quotation and estimate output.



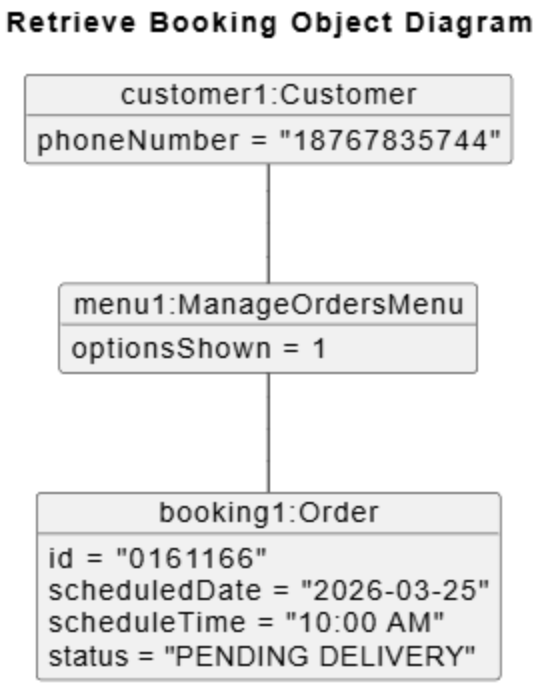
Check Order Status Sequence Diagram

Shows the interaction steps for checking stored order statuses.



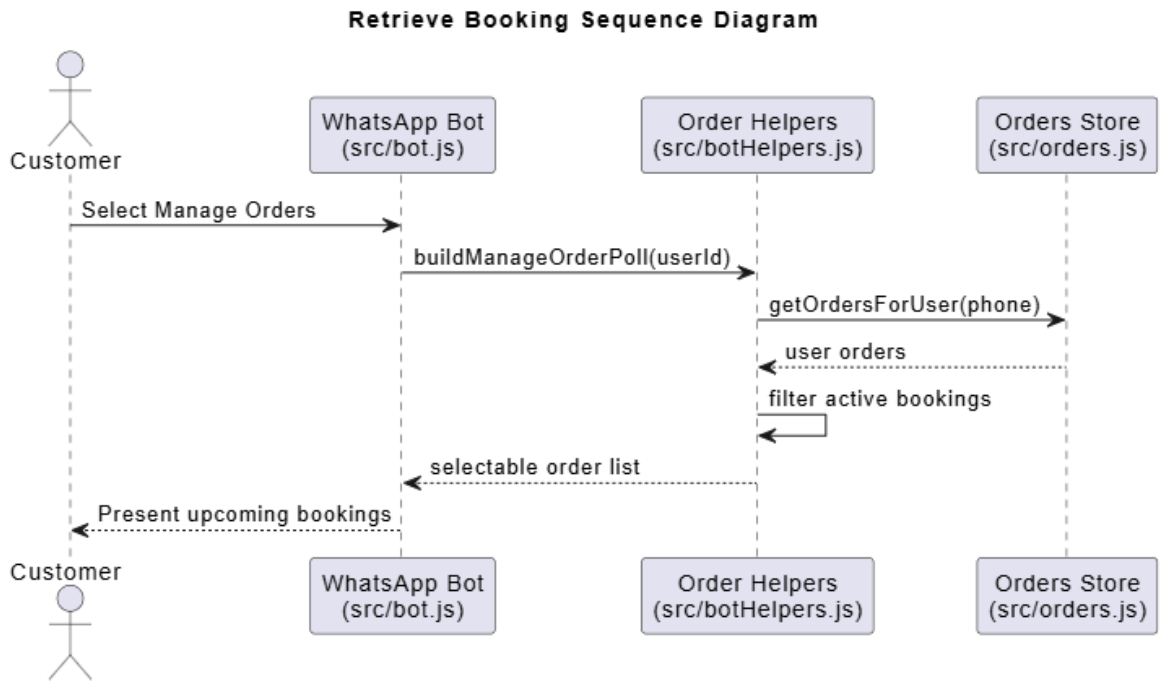
Check Order Status Object Diagram

Shows a runtime snapshot of the customer status lookup result.



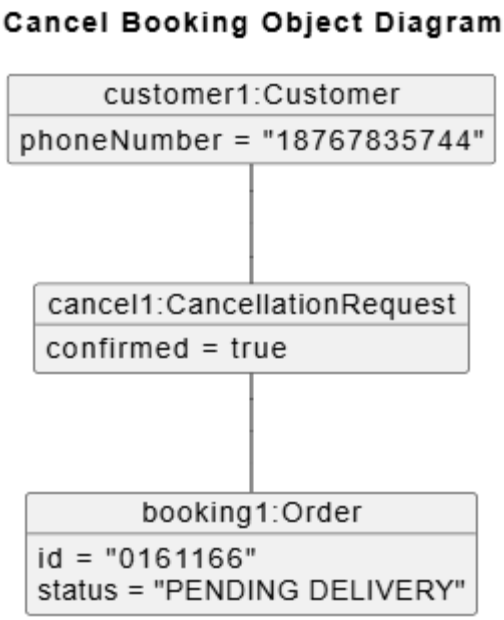
Retrieve Booking Sequence Diagram

Shows how active bookings are retrieved for customer self-service.



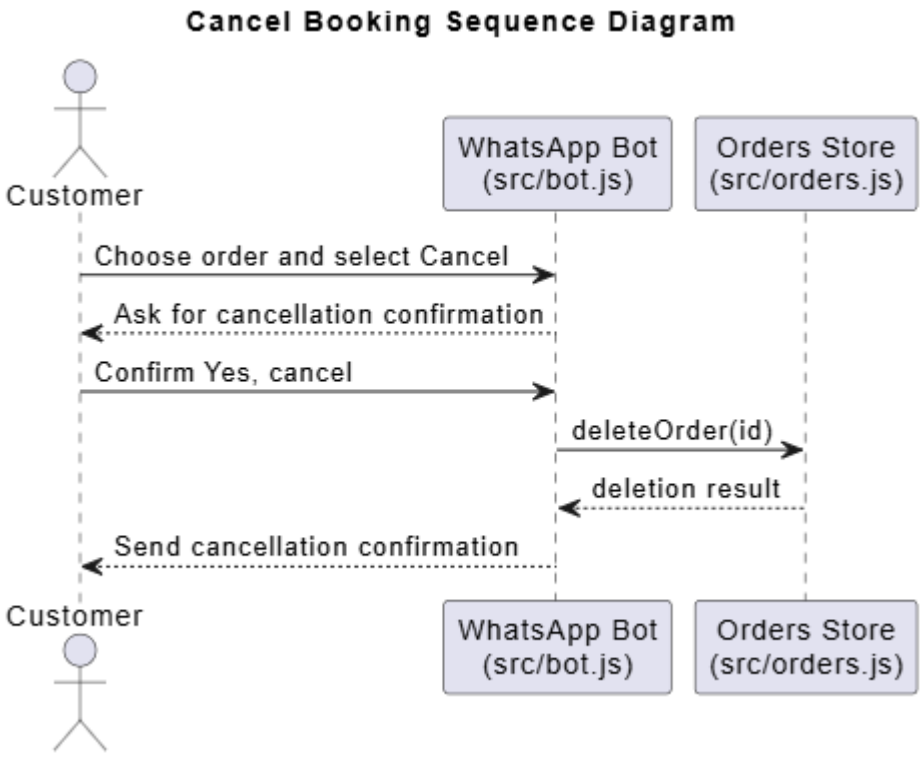
Retrieve Booking Object Diagram

Shows a runtime snapshot of the booking retrieval view.



Cancel Booking Sequence Diagram

Shows the confirmation and removal flow for cancelling a booking.



Cancel Booking Object Diagram

Shows a runtime snapshot of the booking cancellation request.

6 Implementation

6.1 Discussion

The WhatsApp Chatbot Order System for White Rose Interiors was implemented using a lightweight, service-orientated architecture designed to support real-time customer interaction and administrative control.

The backend of the system was developed using **Node.js and Express**, which provide an efficient environment for handling asynchronous requests and managing chatbot interactions. The chatbot functionality was implemented using the **whatsapp-web.js** library, enabling automated communication with customers through WhatsApp.

Data persistence in the system is handled using **local JSON files** (e.g., orders.json, appointments.json, reminders.json). This approach was chosen to simplify development and deployment while still supporting the required functionality for storing and retrieving order, booking, and notification data. Although not as scalable as a database system, it is sufficient for the scope of this project.

The system includes several key functional modules:

- **Quotation and Order Management**, which handles customer requests and stores order details
- **Appointment Scheduling**, which allows customers to book, edit, or cancel appointments
- **Notification System**, which manages reminders, proactive updates, and completion messages
- **Admin Dashboard**, which provides an interface for viewing orders, updating job status, and managing pricing

PDF generation for estimates and invoices is implemented using **PDFKit**, allowing the system to dynamically create documents for customers. Additionally, **Nodemailer** is used to support email-based communication where required.

The admin dashboard was developed using **HTML, CSS, and JavaScript**, providing a simple and accessible interface for administrative tasks such as viewing system data, updating job statuses, and modifying pricing.

Overall, the chosen technologies were selected for their simplicity, flexibility, and compatibility with real-time communication systems. The architecture supports the core use cases of the system while remaining easy to maintain and extend.

6.2 Code Repository

Code Repository

The full implementation of the WhatsApp Chatbot Order System is available in the following GitHub repository:

Repository Link: https://github.com/RICK-MONK/COMP2171-Project